

Lecture 2: Linear Regression from Four Views

Tao LIN

SoE, Westlake University

Fall 2026



1 Regression and Classification

- Regression
- Classification

2 Linear Regression

- Definition of Linear Regression
- Optimization Basics
- Gradient Descent (GD) variants for Linear Regression
- Normal Equations and Least Squares

One model, four connected views:

- **Data:** predictions and residuals
- **Objective:** MSE, MAE, and Huber loss
- **Optimization:** GD and stochastic gradients
- **Geometry:** projection and least squares

Next lecture:

- The probabilistic interpretation of linear regression
- Maximum Likelihood Estimation (MLE)
- Over-fitting and Under-fitting
- Polynomial Regression, Ridge Regression, and Lasso

Reading materials

- Chapter 1, Stanford CS 229 Lecture Notes,
https://cs229.stanford.edu/notes2022fall/main_notes.pdf
- Chapter 3.1, Bishop, Pattern Recognition and Machine Learning
- Dr. Robert M. Gower, Expected smoothness is the key to understanding minibatching for stochastic gradient methods,
https://gowerrobert.github.io/pdf/talks/expected_smoothness.pdf

Reference

- EPFL, CS-433 Machine Learning, https://github.com/epfml/ML_course

Three questions for this lecture

Part 1: From data to an objective

Which line fits the data, and what should count as an important error?

Part 2: From an objective to an algorithm

How does a gradient move the fitted line, and when does optimization struggle?

Part 3: From equations to geometry

What can the data identify when several parameter vectors fit equally well?

Keep returning to the same four views: data, parameters, predictions, and loss.

Table of Contents

1 Regression and Classification

- Regression
- Classification

2 Linear Regression

Table of Contents

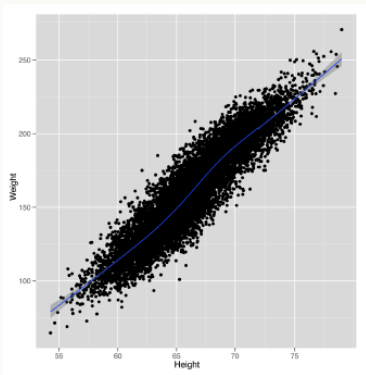
1 Regression and Classification

- Regression
- Classification

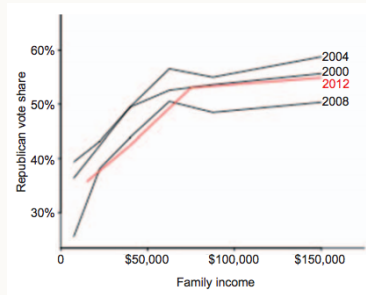
2 Linear Regression

- Definition of Linear Regression
- Optimization Basics
- Gradient Descent (GD) variants for Linear Regression
- Normal Equations and Least Squares

What is regression?

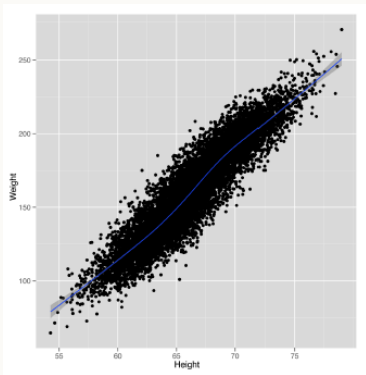


(a) Height is correlated with weight. Taken from "Machine Learning for Hackers"

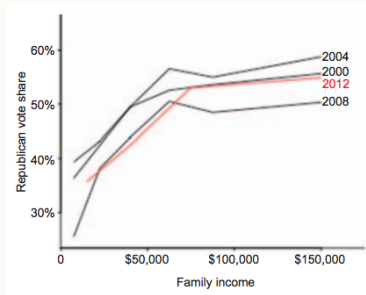


(b) Do rich people vote for republicans? Taken from Avi Feller et. al. 2013, Red state/blue state in 2012 elections.

What is regression?



(a) Height is correlated with weight. Taken from "Machine Learning for Hackers"

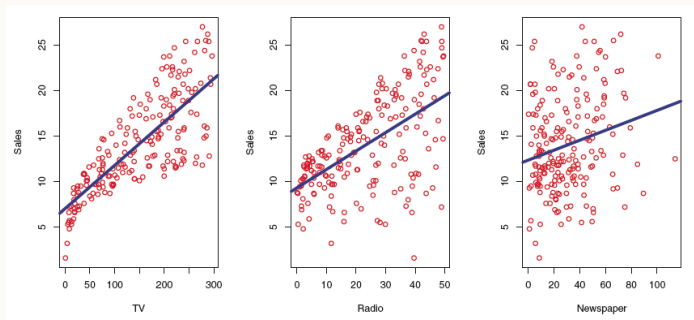


(b) Do rich people vote for republicans? Taken from Avi Feller et. al. 2013, Red state/blue state in 2012 elections.

Regression predicts a numerical response from input variables.

Dataset for regression

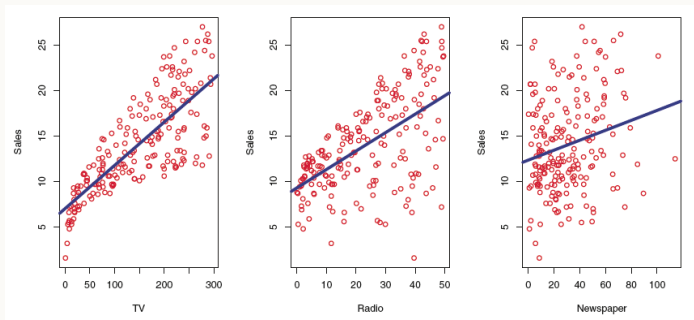
$$\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N \subseteq \mathcal{X} \times \mathcal{Y} \quad (1)$$



Dataset for regression

$$\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N \subseteq \mathcal{X} \times \mathcal{Y} \quad (1)$$

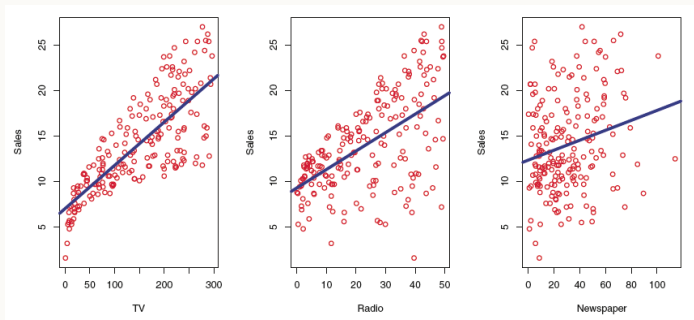
- data consists of pairs $(\mathbf{x}_n, y_n)$, where y_n is the n 'th output and $\mathbf{x}_n$ is a vector of D inputs.



Dataset for regression

$$\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N \subseteq \mathcal{X} \times \mathcal{Y} \quad (1)$$

- **data** consists of pairs $(\mathbf{x}_n, y_n)$, where y_n is the n 'th **output** and $\mathbf{x}_n$ is a vector of D **inputs**.
- The number of pairs N is the **data-size** and D is the **dimensionality**.



Two goals of regression

The regression function approximates the output y_n “well enough” given inputs $\mathbf{x}_n$.

$$y_n \approx f(\mathbf{x}_n), \text{ for all } n \quad (2)$$

Two goals of regression

The regression function approximates the output y_n “well enough” given inputs $\mathbf{x}_n$.

$$y_n \approx f(\mathbf{x}_n), \text{ for all } n \quad (2)$$

- 1 **prediction**: predict outputs for new inputs.

Two goals of regression

The regression function approximates the output y_n “well enough” given inputs $\mathbf{x}_n$.

$$y_n \approx f(\mathbf{x}_n), \text{ for all } n \quad (2)$$

- 1 **prediction**: predict outputs for new inputs.

e.g., what is the weight of a person who is 170 cm tall?

Two goals of regression

The regression function approximates the output y_n “well enough” given inputs $\mathbf{x}_n$.

$$y_n \approx f(\mathbf{x}_n), \text{ for all } n \quad (2)$$

- 1 **prediction**: predict outputs for new inputs.

e.g., what is the weight of a person who is 170 cm tall?

- 2 **interpretation**: describe how inputs and outputs are associated.

Two goals of regression

The regression function approximates the output y_n “well enough” given inputs $\mathbf{x}_n$.

$$y_n \approx f(\mathbf{x}_n), \text{ for all } n \quad (2)$$

- 1 **prediction**: predict outputs for new inputs.

e.g., what is the weight of a person who is 170 cm tall?

- 2 **interpretation**: describe how inputs and outputs are associated.

e.g., are taller people heavier too?

Table of Contents

1 Regression and Classification

- Regression
- Classification

2 Linear Regression

- Definition of Linear Regression
- Optimization Basics
- Gradient Descent (GD) variants for Linear Regression
- Normal Equations and Least Squares

Regression and classification: where is the boundary?

Regression predicts a numerical response. Classification predicts a category:

$$f : \mathcal{X} \longrightarrow \{\mathcal{C}_0, \dots, \mathcal{C}_{K-1}\}.$$

Nominal class labels have no natural numerical ordering or distance.

If we encode *fraud* / *legitimate* as 0/1, have we made this ordinary regression?

Regression and classification: where is the boundary?

Regression predicts a numerical response. Classification predicts a category:

$$f : \mathcal{X} \longrightarrow \{\mathcal{C}_0, \dots, \mathcal{C}_{K-1}\}.$$

Nominal class labels have no natural numerical ordering or distance.

If we encode *fraud* / *legitimate* as 0/1, have we made this ordinary regression?

No. Least squares is a possible baseline, but its scores need not lie in $[0, 1]$; a decision threshold and classification error costs still need to be chosen. Relabeling categories should not invent numerical distances.

Table of Contents

1 Regression and Classification

2 Linear Regression

- Definition of Linear Regression
- Optimization Basics
- Gradient Descent (GD) variants for Linear Regression
- Normal Equations and Least Squares

Table of Contents

1 Regression and Classification

- Regression
- Classification

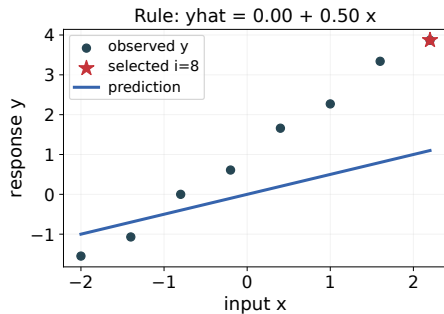
2 Linear Regression

- Definition of Linear Regression
- Optimization Basics
- Gradient Descent (GD) variants for Linear Regression
- Normal Equations and Least Squares

What the experiment shows

Data points $\rightarrow$ model as a rule

Can one rule predict a value at an input we have not observed?



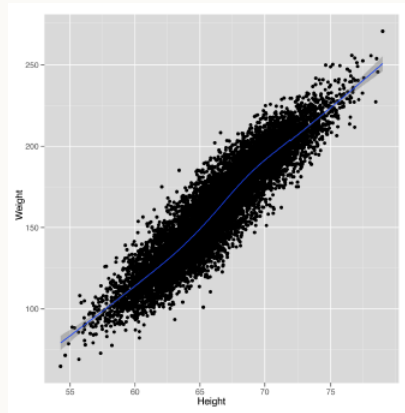
One shared rule
 $\hat{y}(x) = w_0 + w_1 x$

Unseen $x = 0.7$
 $\hat{y} = 0.35$

Conclusion: One parameter pair defines predictions for every input, including unseen x . At $x=0.7$, this rule predicts $\hat{y}=0.35$; the prediction is not an observed label.

Definition

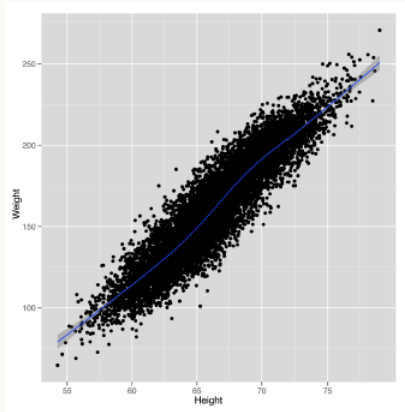
Linear regression is a **model**:



Definition

Linear regression is a **model**:

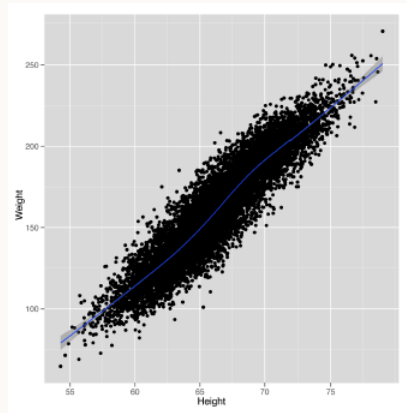
- $y_n \approx f(\mathbf{x}_n)$ for all n



Definition

Linear regression is a **model**:

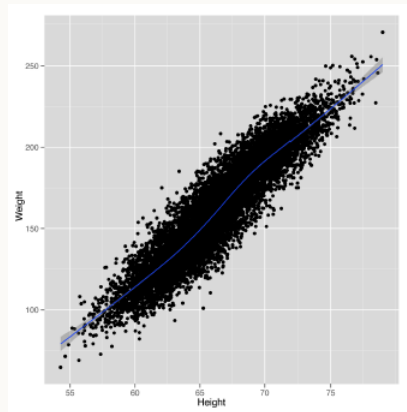
- $y_n \approx f(\mathbf{x}_n)$ for all n
- $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N \subseteq \mathcal{X} \times \mathcal{Y}$



Definition

Linear regression is a **model**:

- $y_n \approx f(\mathbf{x}_n)$ for all n
- $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N \subseteq \mathcal{X} \times \mathcal{Y}$
- a linear relationship is assumed for f



Detailed definition

Simple linear regression (w/ only one input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} \quad (3)$$

Detailed definition

Simple linear regression (w/ only one input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} \quad (3)$$

Here, $\mathbf{w} = (w_0, w_1)$ are the two **parameters** of the model. They describe f .

Multiple linear regression (multiple input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} + \dots + w_D x_{nD} \quad (4)$$

(6)

Detailed definition

Simple linear regression (w/ only one input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} \quad (3)$$

Here, $\mathbf{w} = (w_0, w_1)$ are the two **parameters** of the model. They describe f .

Multiple linear regression (multiple input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} + \dots + w_D x_{nD} \quad (4)$$

$$= w_0 + \mathbf{x}_n^\top \begin{pmatrix} w_1 \\ \vdots \\ w_D \end{pmatrix} \quad (5)$$

$$(6)$$

Detailed definition

Simple linear regression (w/ only one input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} \quad (3)$$

Here, $\mathbf{w} = (w_0, w_1)$ are the two **parameters** of the model. They describe f .

Multiple linear regression (multiple input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} + \dots + w_D x_{nD} \quad (4)$$

$$= w_0 + \mathbf{x}_n^\top \begin{pmatrix} w_1 \\ \vdots \\ w_D \end{pmatrix} \quad (5)$$

$$=: \tilde{\mathbf{x}}_n^\top \tilde{\mathbf{w}} \quad (6)$$

Detailed definition

Simple linear regression (w/ only one input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} \quad (3)$$

Here, $\mathbf{w} = (w_0, w_1)$ are the two **parameters** of the model. They describe f .

Multiple linear regression (multiple input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} + \dots + w_D x_{nD} \quad (4)$$

$$= w_0 + \mathbf{x}_n^\top \begin{pmatrix} w_1 \\ \vdots \\ w_D \end{pmatrix} \quad (5)$$

$$=: \tilde{\mathbf{x}}_n^\top \tilde{\mathbf{w}} \quad (6)$$

We add a tilde over the input vector & weights, to indicate containing the additional offset term (a.k.a. bias term).

Detailed definition

Simple linear regression (w/ only one input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} \quad (3)$$

Here, $\mathbf{w} = (w_0, w_1)$ are the two **parameters** of the model. They describe f .

Multiple linear regression (multiple input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} + \dots + w_D x_{nD} \quad (4)$$

$$= w_0 + \mathbf{x}_n^\top \begin{pmatrix} w_1 \\ \vdots \\ w_D \end{pmatrix} \quad (5)$$

$$=: \tilde{\mathbf{x}}_n^\top \tilde{\mathbf{w}} \quad (6)$$

We add a tilde over the input vector & weights, to indicate containing the additional offset term (a.k.a. bias term).

Goal of “learning/estimation/fitting”

Detailed definition

Simple linear regression (w/ only one input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} \quad (3)$$

Here, $\mathbf{w} = (w_0, w_1)$ are the two **parameters** of the model. They describe f .

Multiple linear regression (multiple input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} + \dots + w_D x_{nD} \quad (4)$$

$$= w_0 + \mathbf{x}_n^\top \begin{pmatrix} w_1 \\ \vdots \\ w_D \end{pmatrix} \quad (5)$$

$$=: \tilde{\mathbf{x}}_n^\top \tilde{\mathbf{w}} \quad (6)$$

We add a tilde over the input vector & weights, to indicate containing the additional offset term (a.k.a. bias term).

Goal of “learning/estimation/fitting”

Given data $\mathcal{D}$, we would like to find $\tilde{\mathbf{w}} = [w_0, w_1, \dots, w_D]$.

Detailed definition

Simple linear regression (w/ only one input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} \quad (3)$$

Here, $\mathbf{w} = (w_0, w_1)$ are the two **parameters** of the model. They describe f .

Multiple linear regression (multiple input dimension):

$$y_n \approx f(\mathbf{x}_n) := w_0 + w_1 x_{n1} + \dots + w_D x_{nD} \quad (4)$$

$$= w_0 + \mathbf{x}_n^\top \begin{pmatrix} w_1 \\ \vdots \\ w_D \end{pmatrix} \quad (5)$$

$$=: \tilde{\mathbf{x}}_n^\top \tilde{\mathbf{w}} \quad (6)$$

We add a tilde over the input vector & weights, to indicate containing the additional offset term (a.k.a. bias term).

Goal of “learning/estimation/fitting”

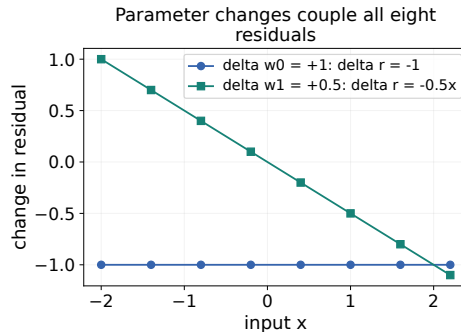
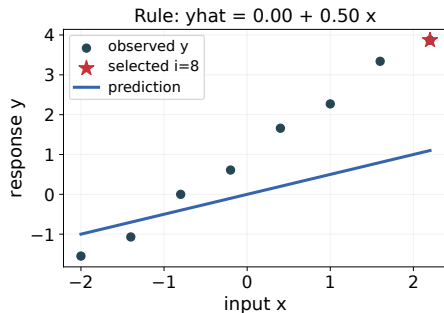
Given data $\mathcal{D}$, we would like to find $\tilde{\mathbf{w}} = [w_0, w_1, \dots, w_D]$.

We need an optimization algorithm!

What the experiment shows

Parameters → coordinated changes

Will changing the intercept move all predictions equally? What changes for slope?

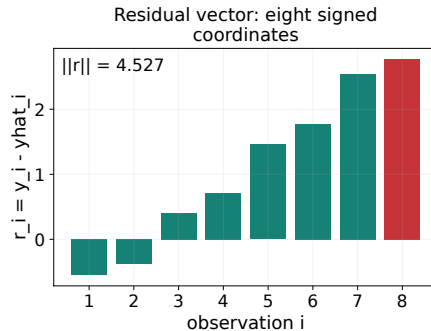
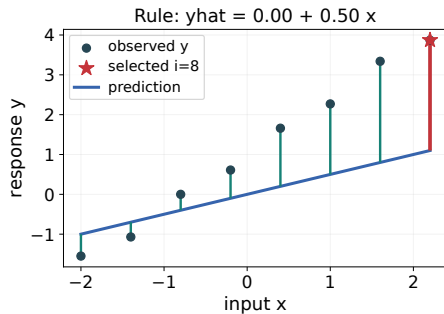


Conclusion: Increasing w_0 by 1 moves every prediction up by 1 and every residual down by 1. Increasing w_1 by 0.5 changes each residual by $-0.5x_i$, with opposite effects on opposite sides of $x=0$.

What the experiment shows

Prediction errors → residual vector

Which vertical gaps are positive, and how do eight gaps become one vector?



Conclusion: Each signed vertical gap is $r_i = y_i - \hat{y}_i$. Stacking gives $r = y - Xw$, an eight-dimensional vector. Opposite signs can cancel in a sum, but cannot erase its nonzero coordinates or norm.

Why learn about *linear* regression?

- simple

Why learn about *linear* regression?

- simple
- easy to understand

Why learn about *linear* regression?

- simple
- easy to understand
- widely used

Why learn about *linear* regression?

- simple
- easy to understand
- widely used
- easily generalized to non-linear models

Why learn about *linear* regression?

- simple
- easy to understand
- widely used
- easily generalized to non-linear models
- we can learn almost all fundamental concepts of ML with regression alone

Table of Contents

1 Regression and Classification

- Regression
- Classification

2 Linear Regression

- Definition of Linear Regression
- **Optimization Basics**
- Gradient Descent (GD) variants for Linear Regression
- Normal Equations and Least Squares

Motivation

Consider the following models.

1-parameter model: $y_n \approx w_0$

2-parameter model: $y_n \approx w_0 + w_1 x_{n1}$

Motivation

Consider the following models.

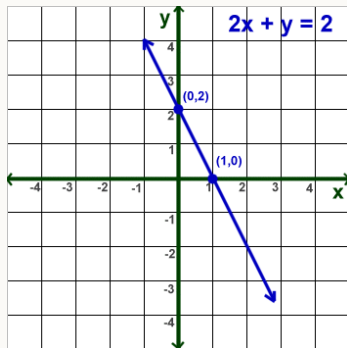
1-parameter model: $y_n \approx w_0$

2-parameter model: $y_n \approx w_0 + w_1 x_{n1}$

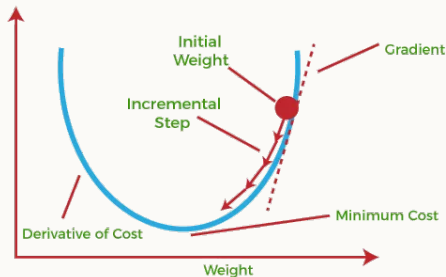
Q: How can we **estimate** values of $\mathbf{w}$ given the data $\mathcal{D}$?

Analytical vs. Numerical solutions

- **Analytical solutions** expose the structure of problems for which a closed form is available.



- **Gradient Descent** (GD) approaches a solution iteratively and scales to many models and datasets.



Residuals to half-MSE

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Turn N signed errors into one nonnegative scalar without cancellation.

Given: $r_i = y_i - \mathbf{x}_i^\top \mathbf{w}$ and $\mathbf{r} = (r_1, \dots, r_N)^\top = \mathbf{y} - \mathbf{X}\mathbf{w}$.

Residuals to half-MSE

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Turn N signed errors into one nonnegative scalar without cancellation.

Given: $r_i = y_i - \mathbf{x}_i^\top \mathbf{w}$ and $\mathbf{r} = (r_1, \dots, r_N)^\top = \mathbf{y} - \mathbf{X}\mathbf{w}$.

STEP 1 · STRUCTURE

Stack, then square:

$$\|\mathbf{r}\|_2^2 = \mathbf{r}^\top \mathbf{r} = \sum_{i=1}^N r_i^2.$$

Every contribution is nonnegative.

Residuals to half-MSE

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Turn N signed errors into one nonnegative scalar without cancellation.

Given: $r_i = y_i - \mathbf{x}_i^\top \mathbf{w}$ and $\mathbf{r} = (r_1, \dots, r_N)^\top = \mathbf{y} - \mathbf{X}\mathbf{w}$.

STEP 1 · STRUCTURE

Stack, then square:

$$\|\mathbf{r}\|_2^2 = \mathbf{r}^\top \mathbf{r} = \sum_{i=1}^N r_i^2.$$

Every contribution is nonnegative.

STEP 2 · FINISH

Average over observations and insert the convenient half:

$$J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{r}\|_2^2 = \frac{1}{2N} \sum_{i=1}^N r_i^2.$$

Residuals to half-MSE

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Turn N signed errors into one nonnegative scalar without cancellation.

Given: $r_i = y_i - \mathbf{x}_i^\top \mathbf{w}$ and $\mathbf{r} = (r_1, \dots, r_N)^\top = \mathbf{y} - \mathbf{X}\mathbf{w}$.

STEP 1 · STRUCTURE

Stack, then square:

$$\|\mathbf{r}\|_2^2 = \mathbf{r}^\top \mathbf{r} = \sum_{i=1}^N r_i^2.$$

Every contribution is nonnegative.

DIMENSION / EDGE CHECK

$\mathbf{r} \in \mathbb{R}^N$ but $J(\mathbf{w}) \in \mathbb{R}$. If $\mathbf{r} \mapsto c\mathbf{r}$, then $J \mapsto c^2 J$.

STEP 2 · FINISH

Average over observations and insert the convenient half:

$$J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{r}\|_2^2 = \frac{1}{2N} \sum_{i=1}^N r_i^2.$$

COMMON TRAP

Using $\sum_i r_i$ lets positive and negative errors cancel; using $1/N$ rather than $1/(2N)$ changes the gradient by a factor of two.

Residuals to half-MSE

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Turn N signed errors into one nonnegative scalar without cancellation.

Given: $r_i = y_i - \mathbf{x}_i^\top \mathbf{w}$ and $\mathbf{r} = (r_1, \dots, r_N)^\top = \mathbf{y} - \mathbf{X}\mathbf{w}$.

STEP 1 · STRUCTURE

Stack, then square:

$$\|\mathbf{r}\|_2^2 = \mathbf{r}^\top \mathbf{r} = \sum_{i=1}^N r_i^2.$$

Every contribution is nonnegative.

DIMENSION / EDGE CHECK

$\mathbf{r} \in \mathbb{R}^N$ but $J(\mathbf{w}) \in \mathbb{R}$. If $\mathbf{r} \mapsto c\mathbf{r}$, then $J \mapsto c^2 J$.

EXPLAIN IN ONE SENTENCE

Square to prevent cancellation, average to compare datasets, and use $1/2$ to simplify the gradient.

STEP 2 · FINISH

Average over observations and insert the convenient half:

$$J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{r}\|_2^2 = \frac{1}{2N} \sum_{i=1}^N r_i^2.$$

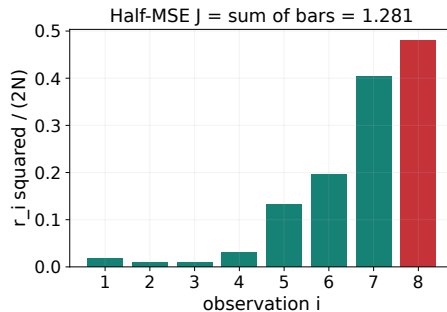
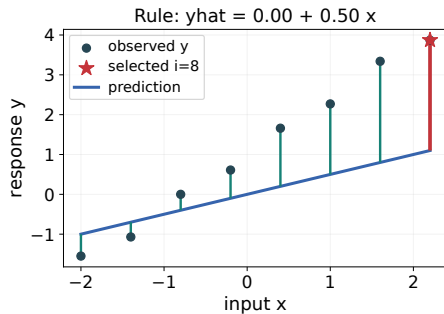
COMMON TRAP

Using $\sum_i r_i$ lets positive and negative errors cancel; using $1/N$ rather than $1/(2N)$ changes the gradient by a factor of two.

What the experiment shows

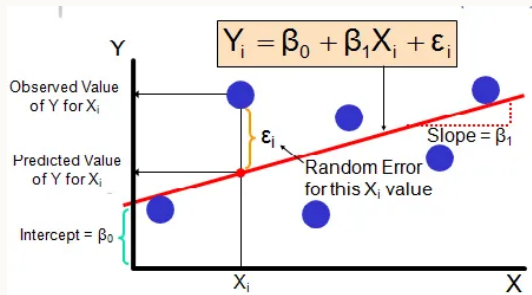
Residual vector $\rightarrow$ scalar objective

How can we score a whole residual vector without positive and negative errors canceling?



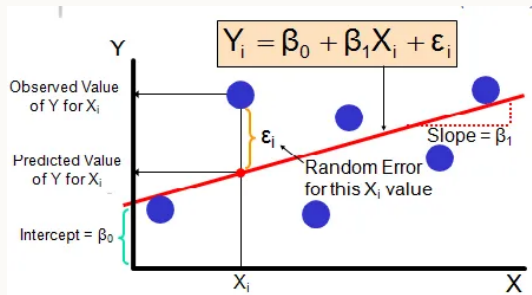
Conclusion: Each bar is $r_i^2 / 16$. Their sum is $J = ||r||^2 / (2N)$, $N=8$. This scalar keeps error magnitude without signed cancellation; doubling every residual quadruples J.

Approximate optimal solution w^* : cost function



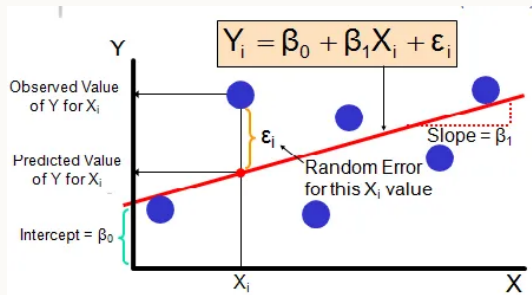
- The cost function quantifies the difference between the model's predictions and the actual data.

Approximate optimal solution w^* : cost function



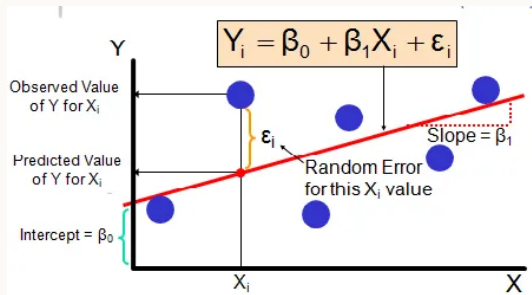
- The cost function quantifies the difference between the model's predictions and the actual data.
- Loss design is a modeling choice.

Approximate optimal solution w^* : cost function



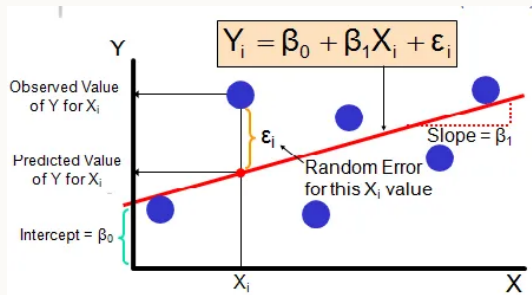
- The cost function quantifies the difference between the model's predictions and the actual data.
- Loss design is a modeling choice. **What behavior do we want?**

Approximate optimal solution w^* : cost function



- The cost function quantifies the difference between the model's predictions and the actual data.
- Loss design is a modeling choice. **What behavior do we want?**
 - Should positive and negative residuals be treated symmetrically?

Approximate optimal solution w^* : cost function

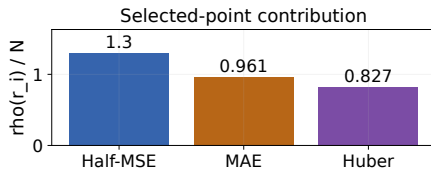
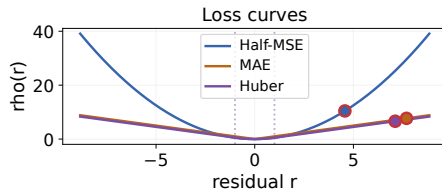
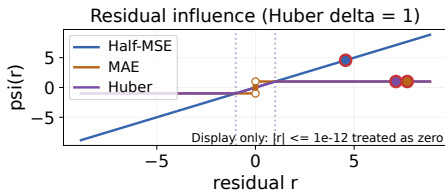
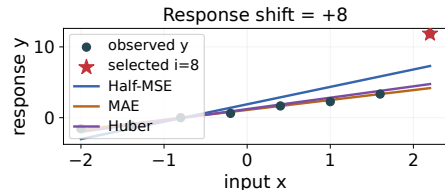


- The cost function quantifies the difference between the model's predictions and the actual data.
- Loss design is a modeling choice. **What behavior do we want?**
 - Should positive and negative residuals be treated symmetrically?
 - How strongly should a very large residual influence the fit?

What the experiment shows

Loss and influence → fitted lines

Move the last response upward. Which fitted line moves most, and why?



Conclusion: The +8 response contamination pulls half-MSE most: $\psi(r)=r$ is unbounded. MAE and Huber have bounded residual influence. Outlined dots track the SAME observation at each loss's fitted line; raw losses have different scales.

Mean Squared Error (MSE) and outliers

$$\text{MSE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N [y_n - f_{\mathbf{w}}(\mathbf{x}_n)]^2, \quad J = \text{MSE} / 2 \quad (7)$$

Mean Squared Error (MSE) and outliers

$$\text{MSE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N [y_n - f_{\mathbf{w}}(\mathbf{x}_n)]^2, \quad J = \text{MSE} / 2 \quad (7)$$

How does MSE answer our two design questions?

Mean Squared Error (MSE) and outliers

$$\text{MSE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N [y_n - f_{\mathbf{w}}(\mathbf{x}_n)]^2, \quad J = \text{MSE} / 2 \quad (7)$$

How does MSE answer our two design questions? It is symmetric, and its penalty grows quadratically with $|r_n|$.

Mean Squared Error (MSE) and outliers

$$\text{MSE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N [y_n - f_{\mathbf{w}}(\mathbf{x}_n)]^2, \quad J = \text{MSE} / 2 \quad (7)$$

How does MSE answer our two design questions? It is symmetric, and its penalty grows quadratically with $|r_n|$.

Definition 1 (Outliers)

Outliers are data examples that are far away from most of the other examples.

Mean Squared Error (MSE) and outliers

$$\text{MSE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N [y_n - f_{\mathbf{w}}(\mathbf{x}_n)]^2, \quad J = \text{MSE} / 2 \quad (7)$$

How does MSE answer our two design questions? It is symmetric, and its penalty grows quadratically with $|r_n|$.

Definition 1 (Outliers)

Outliers are data examples that are far away from most of the other examples.

MSE can be strongly influenced by outliers.

Mean Squared Error (MSE) and outliers

$$\text{MSE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N [y_n - f_{\mathbf{w}}(\mathbf{x}_n)]^2, \quad J = \text{MSE} / 2 \quad (7)$$

How does MSE answer our two design questions? It is symmetric, and its penalty grows quadratically with $|r_n|$.

Definition 1 (Outliers)

Outliers are data examples that are far away from most of the other examples.

MSE can be strongly influenced by outliers. **Why?**

Mean Squared Error (MSE) and outliers

$$\text{MSE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N [y_n - f_{\mathbf{w}}(\mathbf{x}_n)]^2, \quad J = \text{MSE} / 2 \quad (7)$$

How does MSE answer our two design questions? It is symmetric, and its penalty grows quadratically with $|r_n|$.

Definition 1 (Outliers)

Outliers are data examples that are far away from most of the other examples.

MSE can be strongly influenced by outliers. **Why?**

- Squaring gives large residuals disproportionately large weight.

Mean Squared Error (MSE) and outliers

$$\text{MSE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N [y_n - f_{\mathbf{w}}(\mathbf{x}_n)]^2, \quad J = \text{MSE} / 2 \quad (7)$$

How does MSE answer our two design questions? It is symmetric, and its penalty grows quadratically with $|r_n|$.

Definition 1 (Outliers)

Outliers are data examples that are far away from most of the other examples.

MSE can be strongly influenced by outliers. **Why?**

- Squaring gives large residuals disproportionately large weight.
- This may be desirable when large errors are genuinely costly, but harmful when they arise from contamination or noise.

Mean Absolute Error (MAE)

Handling outliers well is a desired *statistical* property.

Mean Absolute Error (MAE)

Handling outliers well is a desired *statistical* property.

$$\text{MAE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N |y_n - f_{\mathbf{w}}(\mathbf{x}_n)| \quad (8)$$

Mean Absolute Error (MAE)

Handling outliers well is a desired *statistical* property.

$$\text{MAE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N |y_n - f_{\mathbf{w}}(\mathbf{x}_n)| \quad (8)$$

+ MAE bounds residual influence; high-leverage inputs can still matter.

Mean Absolute Error (MAE)

Handling outliers well is a desired *statistical* property.

$$\text{MAE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N |y_n - f_{\mathbf{w}}(\mathbf{x}_n)| \quad (8)$$

- + MAE bounds residual influence; high-leverage inputs can still matter.
- MAE is not differentiable at zero.

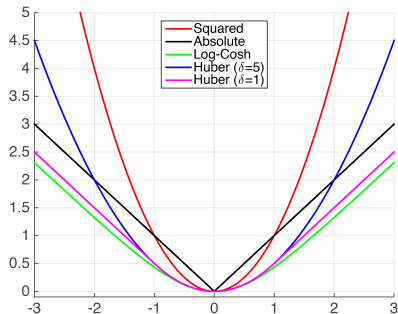
Discussion on different cost functions

Define the residual $r_n := y_n - f_{\mathbf{w}}(\mathbf{x}_n)$. Then

$$\text{MSE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N r_n^2 \quad (\text{MSE})$$

$$\text{MAE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N |r_n| \quad (\text{MAE})$$

$$\text{Huber}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N \begin{cases} \frac{1}{2} r_n^2 & \text{if } |r_n| \leq \delta, \\ \delta (|r_n| - \frac{1}{2} \delta) & \text{if } |r_n| > \delta, \end{cases} \quad (\text{Huber})$$



Discussion on different cost functions

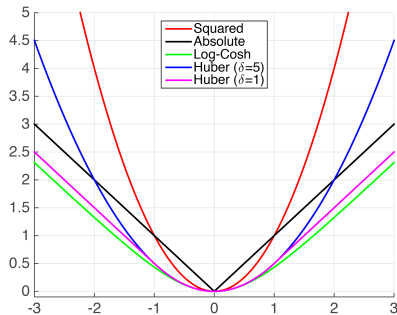
Define the residual $r_n := y_n - f_{\mathbf{w}}(\mathbf{x}_n)$. Then

$$\text{MSE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N r_n^2 \quad (\text{MSE})$$

$$\text{MAE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N |r_n| \quad (\text{MAE})$$

$$\text{Huber}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N \begin{cases} \frac{1}{2} r_n^2 & \text{if } |r_n| \leq \delta, \\ \delta (|r_n| - \frac{1}{2} \delta) & \text{if } |r_n| > \delta, \end{cases} \quad (\text{Huber})$$

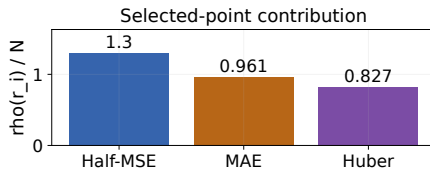
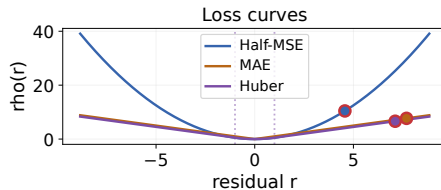
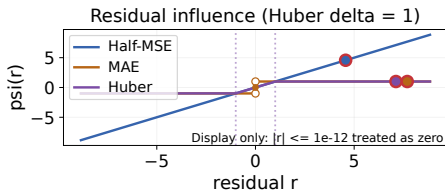
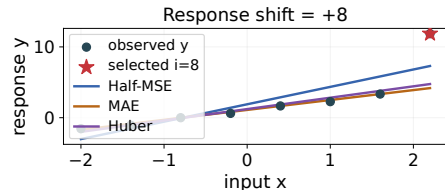
Use $J = \text{MSE} / 2$ in optimization. Huber is quadratic near zero and linear outside.



What the experiment shows

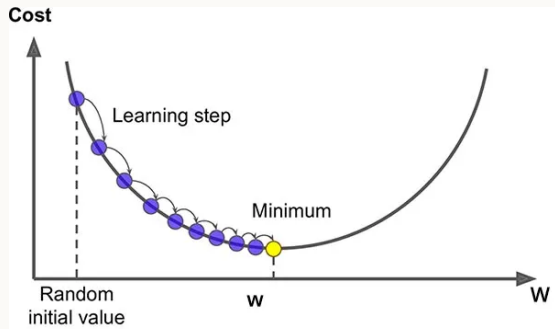
New noise situation → loss choice

Which loss would you defend for occasional corrupted sensor responses?

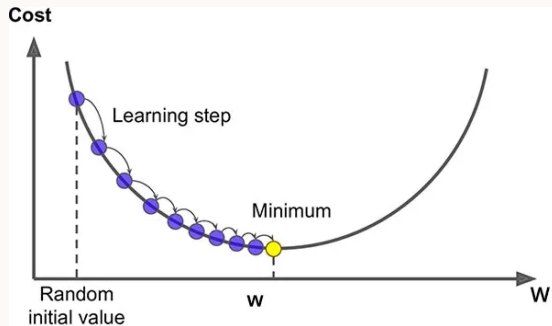


Conclusion: For occasional response corruption, Huber preserves quadratic behavior near zero and limits large residual influence. Set delta using ordinary noise scale; check feature leverage. Gaussian noise supports squared loss; Laplace noise supports absolute loss.

Learning / Estimation / Fitting

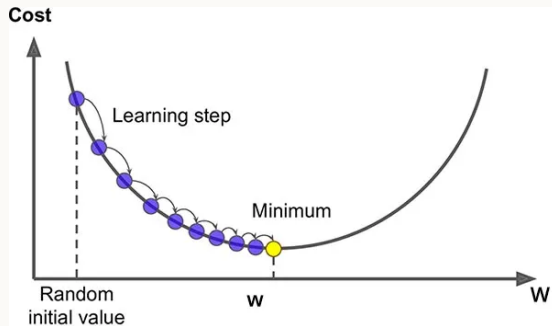


Learning / Estimation / Fitting



Definition 2 (*Learning* problem can be formulated as **optimization problem**)

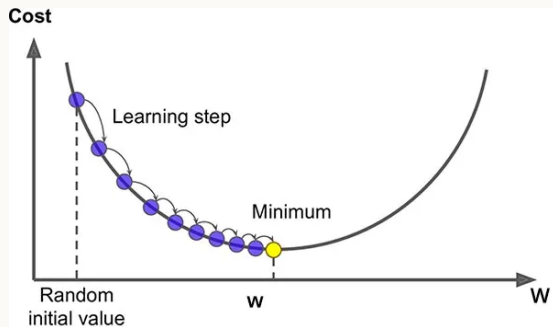
Given a cost function $J(\mathbf{w})$, we wish to find $\mathbf{w}^*$ which minimizes the cost:



Definition 2 (*Learning* problem can be formulated as **optimization problem**)

Given a cost function $J(\mathbf{w})$, we wish to find $\mathbf{w}^*$ which minimizes the cost:

$$\min_{\mathbf{w}} J(\mathbf{w}) \quad \text{subject to } \mathbf{w} \in \mathbb{R}^D \quad (9)$$



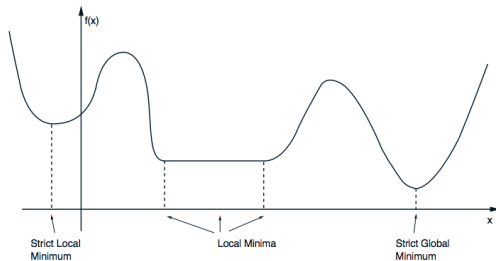
Definition 2 (*Learning* problem can be formulated as **optimization problem**)

Given a cost function $J(\mathbf{w})$, we wish to find $\mathbf{w}^*$ which minimizes the cost:

$$\min_{\mathbf{w}} J(\mathbf{w}) \quad \text{subject to } \mathbf{w} \in \mathbb{R}^D \quad (9)$$

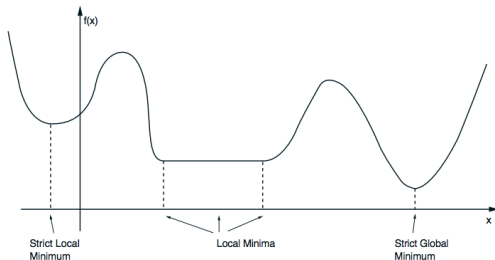
We will use an **optimization algorithm** like GD to solve the problem (to find a good $\mathbf{w}$).

Optimization landscapes



The above figure is taken from Bertsekas, Nonlinear programming.

Optimization landscapes

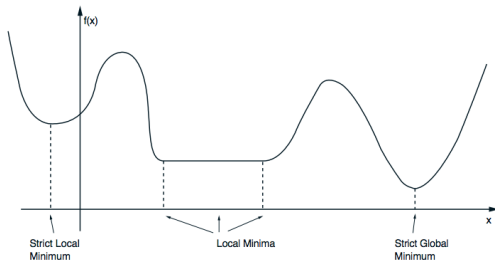


The above figure is taken from Bertsekas, Nonlinear programming.

- A vector $\mathbf{w}^*$ is a **local minimum** of J if it is no worse than its neighbors; i.e. there exists an $\epsilon > 0$ such that,

$$J(\mathbf{w}^*) \leq J(\mathbf{w}), \quad \forall \mathbf{w} \text{ with } \|\mathbf{w} - \mathbf{w}^*\| < \epsilon$$

Optimization landscapes



The above figure is taken from Bertsekas, Nonlinear programming.

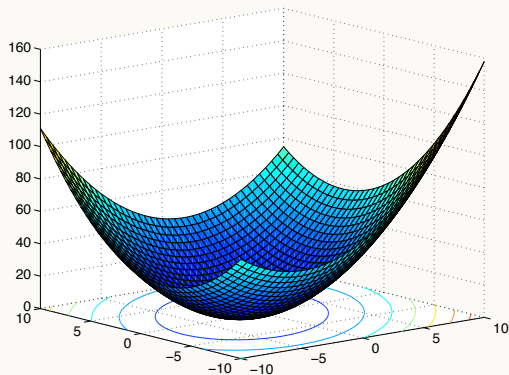
- A vector $\mathbf{w}^*$ is a **local minimum** of J if it is no worse than its neighbors; i.e. there exists an $\epsilon > 0$ such that,

$$J(\mathbf{w}^*) \leq J(\mathbf{w}), \quad \forall \mathbf{w} \text{ with } \|\mathbf{w} - \mathbf{w}^*\| < \epsilon$$

- A vector $\mathbf{w}^*$ is a **global minimum** of J if it is no worse than all others,

$$J(\mathbf{w}^*) \leq J(\mathbf{w}), \quad \forall \mathbf{w} \in \mathbb{R}^D$$

The landscape for linear regression with MSE



- The half-MSE objective $J = \text{MSE} / 2$ is a convex quadratic in $\mathbf{w}$.
- Every local minimum is global.
- Rank deficiency can create many minimizers, but no spurious local minima.

Today, GD and least squares solve the *same* optimization problem from two views.

Half-MSE to its gradient

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Differentiate through the residual while keeping the sign and dimensions visible.

Given: $J(\mathbf{w}) = \mathbf{r}^\top \mathbf{r} / (2N)$, $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w}$, and $\mathbf{X} \in \mathbb{R}^{N \times D}$.

Half-MSE to its gradient

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Differentiate through the residual while keeping the sign and dimensions visible.

Given: $J(\mathbf{w}) = \mathbf{r}^\top \mathbf{r} / (2N)$, $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w}$, and $\mathbf{X} \in \mathbb{R}^{N \times D}$.

STEP 1 · STRUCTURE

Differentiate the residual first:

$$d\mathbf{r} = -\mathbf{X}d\mathbf{w}.$$

Then $d(\mathbf{r}^\top \mathbf{r}) = 2\mathbf{r}^\top d\mathbf{r}$.

Half-MSE to its gradient

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Differentiate through the residual while keeping the sign and dimensions visible.

Given: $J(\mathbf{w}) = \mathbf{r}^\top \mathbf{r} / (2N)$, $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w}$, and $\mathbf{X} \in \mathbb{R}^{N \times D}$.

STEP 1 · STRUCTURE

Differentiate the residual first:

$$d\mathbf{r} = -\mathbf{X}d\mathbf{w}.$$

Then $d(\mathbf{r}^\top \mathbf{r}) = 2\mathbf{r}^\top d\mathbf{r}$.

STEP 2 · FINISH

Substitute and expose the coefficient of $d\mathbf{w}$:

$$dJ = -\frac{1}{N}\mathbf{r}^\top \mathbf{X}d\mathbf{w} = \left[-\frac{1}{N}\mathbf{X}^\top \mathbf{r} \right]^\top d\mathbf{w}.$$

Hence $\nabla J(\mathbf{w}) = \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) / N$.

Half-MSE to its gradient

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Differentiate through the residual while keeping the sign and dimensions visible.

Given: $J(\mathbf{w}) = \mathbf{r}^\top \mathbf{r} / (2N)$, $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w}$, and $\mathbf{X} \in \mathbb{R}^{N \times D}$.

STEP 1 · STRUCTURE

Differentiate the residual first:

$$d\mathbf{r} = -\mathbf{X}d\mathbf{w}.$$

Then $d(\mathbf{r}^\top \mathbf{r}) = 2\mathbf{r}^\top d\mathbf{r}$.

DIMENSION / EDGE CHECK

$\mathbf{X}^\top \mathbf{r}$ has shape $D \times 1$, matching $\mathbf{w}$. At $\mathbf{r} = \mathbf{0}$, the gradient is $\mathbf{0}$.

STEP 2 · FINISH

Substitute and expose the coefficient of $d\mathbf{w}$:

$$dJ = -\frac{1}{N} \mathbf{r}^\top \mathbf{X} d\mathbf{w} = \left[-\frac{1}{N} \mathbf{X}^\top \mathbf{r} \right]^\top d\mathbf{w}.$$

Hence $\nabla J(\mathbf{w}) = \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) / N$.

COMMON TRAP

Forgetting $d\mathbf{r} = -\mathbf{X}d\mathbf{w}$ reverses the sign; forgetting the half leaves an unnecessary factor of two.

Half-MSE to its gradient

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Differentiate through the residual while keeping the sign and dimensions visible.

Given: $J(\mathbf{w}) = \mathbf{r}^\top \mathbf{r} / (2N)$, $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w}$, and $\mathbf{X} \in \mathbb{R}^{N \times D}$.

STEP 1 · STRUCTURE

Differentiate the residual first:

$$d\mathbf{r} = -\mathbf{X}d\mathbf{w}.$$

Then $d(\mathbf{r}^\top \mathbf{r}) = 2\mathbf{r}^\top d\mathbf{r}$.

DIMENSION / EDGE CHECK

$\mathbf{X}^\top \mathbf{r}$ has shape $D \times 1$, matching $\mathbf{w}$. At $\mathbf{r} = \mathbf{0}$, the gradient is 0.

EXPLAIN IN ONE SENTENCE

The gradient is the residual pulled back from observation space to parameter space by $\mathbf{X}^\top$.

STEP 2 · FINISH

Substitute and expose the coefficient of $d\mathbf{w}$:

$$dJ = -\frac{1}{N} \mathbf{r}^\top \mathbf{X} d\mathbf{w} = \left[-\frac{1}{N} \mathbf{X}^\top \mathbf{r} \right]^\top d\mathbf{w}.$$

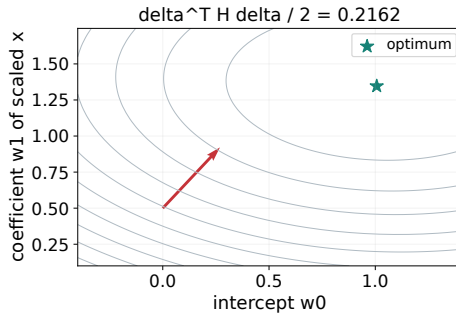
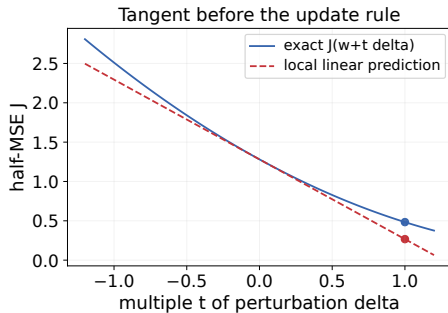
Hence $\nabla J(\mathbf{w}) = \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) / N$.
COMMON TRAP

Forgetting $d\mathbf{r} = -\mathbf{X}d\mathbf{w}$ reverses the sign; forgetting the half leaves an unnecessary factor of two.

What the experiment shows

Directional perturbation $\rightarrow$ local approximation

If we halve a small parameter perturbation, what happens to the tangent approximation error?



Conclusion: The gradient gives the local linear change. The exact quadratic error is $\delta^T H \delta / 2$, so halving δ quarters the approximation error.

Smooth optimization: follow the gradient

Smooth optimization: follow the gradient

Definition 3 (Gradient)

The gradient $\nabla J(\mathbf{w})$ collects the partial derivatives:

Smooth optimization: follow the gradient

Definition 3 (Gradient)

The gradient $\nabla J(\mathbf{w})$ collects the partial derivatives:

$$\nabla J(\mathbf{w}) := \left[\frac{\partial J(\mathbf{w})}{\partial w_1}, \dots, \frac{\partial J(\mathbf{w})}{\partial w_D} \right]^\top \in \mathbb{R}^D, \quad (10)$$

Smooth optimization: follow the gradient

Definition 3 (Gradient)

The gradient $\nabla J(\mathbf{w})$ collects the partial derivatives:

$$\nabla J(\mathbf{w}) := \left[\frac{\partial J(\mathbf{w})}{\partial w_1}, \dots, \frac{\partial J(\mathbf{w})}{\partial w_D} \right]^\top \in \mathbb{R}^D, \quad (10)$$

It is the direction of steepest local increase, and gives the first-order approximation $J(\mathbf{w} + \boldsymbol{\delta}) \approx J(\mathbf{w}) + \nabla J(\mathbf{w})^\top \boldsymbol{\delta}$.

Smooth optimization: follow the gradient

Definition 3 (Gradient)

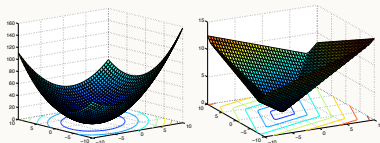
The gradient $\nabla J(\mathbf{w})$ collects the partial derivatives:

$$\nabla J(\mathbf{w}) := \left[\frac{\partial J(\mathbf{w})}{\partial w_1}, \dots, \frac{\partial J(\mathbf{w})}{\partial w_D} \right]^\top \in \mathbb{R}^D, \quad (10)$$

It is the direction of steepest local increase, and gives the first-order approximation $J(\mathbf{w} + \delta) \approx J(\mathbf{w}) + \nabla J(\mathbf{w})^\top \delta$.

For a 2-parameter model, $\text{MSE}(\mathbf{w}) = 2J(\mathbf{w})$ and $\text{MAE}(\mathbf{w})$ are shown below.

(We used $\mathbf{y}_n \approx w_0 + w_1 x_{n1}$ with $\mathbf{y}^\top = [2, -1, 1.5]$ and $\mathbf{x}^\top = [-1, 1, -1]$).



Gradient Descent (GD)

Definition 4 (Gradient Descent)

To minimize the cost function, we iteratively take a step in the (opposite) direction of the gradient

Gradient Descent (GD)

Definition 4 (Gradient Descent)

To minimize the cost function, we iteratively take a step in the (opposite) direction of the gradient

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \nabla J(\mathbf{w}^{(t)}) \quad (11)$$

Gradient Descent (GD)

Definition 4 (Gradient Descent)

To minimize the cost function, we iteratively take a step in the (opposite) direction of the gradient

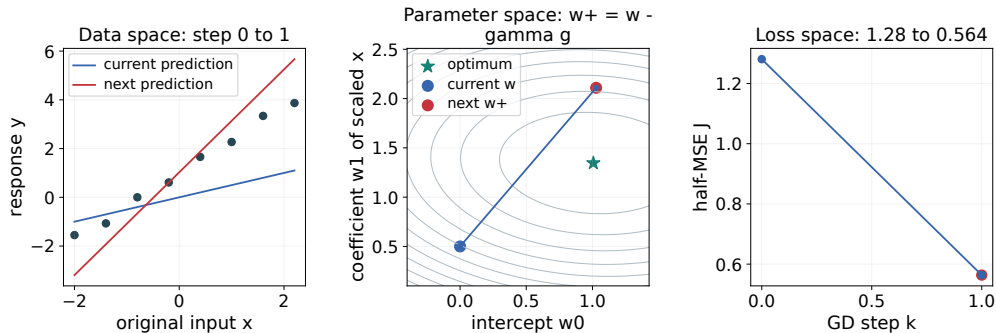
$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \nabla J(\mathbf{w}^{(t)}) \quad (11)$$

where $\gamma > 0$ is the **step-size** (or **learning rate**). Then repeat with the next t .

What the experiment shows

One gradient step $\rightarrow$ three linked spaces

How does one gradient step change the fitted line, parameter point, and loss together?



Conclusion: One update changes the parameter point, fitted line, and half-MSE together: $w+ = w - \text{gamma } g$; $\text{prediction}+ = Xw+$; $J+ = ||y-Xw+||^2 / (2N)$. Blue is current; red is next.

Table of Contents

1 Regression and Classification

- Regression
- Classification

2 Linear Regression

- Definition of Linear Regression
- Optimization Basics
- Gradient Descent (GD) variants for Linear Regression
- Normal Equations and Least Squares

Gradient Descent for linear regression with MSE

Considering a dataset $\mathcal{D} = \{\mathbf{X}, \mathbf{y}\}$ and learnable weights $\mathbf{w} \in \mathbb{R}^D$ for $f_{\mathbf{w}}(\mathbf{X}) = \mathbf{X}\mathbf{w}$.

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N, \quad \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1D} \\ x_{21} & x_{22} & \dots & x_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{ND} \end{bmatrix} \in \mathbb{R}^{N \times D} \quad (12)$$

Gradient Descent for linear regression with MSE

Considering a dataset $\mathcal{D} = \{\mathbf{X}, \mathbf{y}\}$ and learnable weights $\mathbf{w} \in \mathbb{R}^D$ for $f_{\mathbf{w}}(\mathbf{X}) = \mathbf{X}\mathbf{w}$.

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N, \quad \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1D} \\ x_{21} & x_{22} & \dots & x_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{ND} \end{bmatrix} \in \mathbb{R}^{N \times D} \quad (12)$$

We define the residual vector $\mathbf{r}$:

$$\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w} = \begin{pmatrix} r_1 \\ \vdots \\ r_n \\ \vdots \\ r_N \end{pmatrix} \in \mathbb{R}^N, \quad (13)$$

where $r_n := y_n - \mathbf{x}_n^\top \mathbf{w}$.

Gradient Descent for linear regression with MSE

Considering a dataset $\mathcal{D} = \{\mathbf{X}, \mathbf{y}\}$ and learnable weights $\mathbf{w} \in \mathbb{R}^D$ for $f_{\mathbf{w}}(\mathbf{X}) = \mathbf{X}\mathbf{w}$.

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N, \quad \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1D} \\ x_{21} & x_{22} & \dots & x_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{ND} \end{bmatrix} \in \mathbb{R}^{N \times D} \quad (12)$$

We define the residual vector $\mathbf{r}$:

$$\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w} = \begin{pmatrix} r_1 \\ \vdots \\ r_n \\ \vdots \\ r_N \end{pmatrix} \in \mathbb{R}^N, \quad (13)$$

where $r_n := y_n - \mathbf{x}_n^\top \mathbf{w}$. For convenient gradients, use the scaled-MSE objective

$$J(\mathbf{w}) := \frac{1}{2} \text{MSE}(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2,$$

Gradient Descent for linear regression with MSE

Considering a dataset $\mathcal{D} = \{\mathbf{X}, \mathbf{y}\}$ and learnable weights $\mathbf{w} \in \mathbb{R}^D$ for $f_{\mathbf{w}}(\mathbf{X}) = \mathbf{X}\mathbf{w}$.

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N, \quad \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1D} \\ x_{21} & x_{22} & \dots & x_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{ND} \end{bmatrix} \in \mathbb{R}^{N \times D} \quad (12)$$

We define the residual vector $\mathbf{r}$:

$$\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w} = \begin{pmatrix} r_1 \\ \vdots \\ r_n \\ \vdots \\ r_N \end{pmatrix} \in \mathbb{R}^N, \quad (13)$$

where $r_n := y_n - \mathbf{x}_n^\top \mathbf{w}$. For convenient gradients, use the scaled-MSE objective $J(\mathbf{w}) := \frac{1}{2} \text{MSE}(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 = \frac{1}{2N} \mathbf{r}^\top \mathbf{r}$, and then the gradient is given by

$$\nabla J(\mathbf{w}) = -\frac{1}{N} \mathbf{X}^\top \mathbf{r}$$

Eigenmode stability

BOARD DERIVATION

3 minutes · derive, check, explain

Question: For a concrete two-direction quadratic, which learning rates shrink both errors?

Given: $\mathbf{H} = \text{diag}(1, 4)$, $\mathbf{w}^* = \mathbf{0}$, and $\boldsymbol{\eta}_0 = (1, 1)^\top$. The second direction has four times the curvature.

Eigenmode stability

BOARD DERIVATION

3 minutes · derive, check, explain

Question: For a concrete two-direction quadratic, which learning rates shrink both errors?

Given: $\mathbf{H} = \text{diag}(1, 4)$, $\mathbf{w}^* = \mathbf{0}$, and $\boldsymbol{\eta}_0 = (1, 1)^\top$. The second direction has four times the curvature.

STEP 1 · STRUCTURE

GD acts independently on the two coordinates:

$$\boldsymbol{\eta}_{k+1} = (\mathbf{I} - \gamma \mathbf{H}) \boldsymbol{\eta}_k, \quad \begin{cases} a_{1,k+1} = (1 - \gamma) a_{1,k}, \\ a_{2,k+1} = (1 - 4\gamma) a_{2,k}. \end{cases}$$

Eigenmode stability

BOARD DERIVATION

3 minutes · derive, check, explain

Question: For a concrete two-direction quadratic, which learning rates shrink both errors?

Given: $\mathbf{H} = \text{diag}(1, 4)$, $\mathbf{w}^* = \mathbf{0}$, and $\boldsymbol{\eta}_0 = (1, 1)^\top$. The second direction has four times the curvature.

STEP 1 · STRUCTURE

GD acts independently on the two coordinates:

$$\boldsymbol{\eta}_{k+1} = (\mathbf{I} - \gamma \mathbf{H}) \boldsymbol{\eta}_k, \quad \begin{cases} a_{1,k+1} = (1 - \gamma)a_{1,k}, \\ a_{2,k+1} = (1 - 4\gamma)a_{2,k}. \end{cases}$$

STEP 2 · FINISH

Compare three rates:

γ	$1 - \gamma$	$1 - 4\gamma$	behavior
0.4	0.6	-0.6	both shrink
0.5	0.5	-1	boundary
0.6	0.4	-1.4	mode 2 grows

Eigenmode stability

BOARD DERIVATION

3 minutes · derive, check, explain

Question: For a concrete two-direction quadratic, which learning rates shrink both errors?

Given: $\mathbf{H} = \text{diag}(1, 4)$, $\mathbf{w}^* = \mathbf{0}$, and $\boldsymbol{\eta}_0 = (1, 1)^\top$. The second direction has four times the curvature.

STEP 1 · STRUCTURE

GD acts independently on the two coordinates:

$$\boldsymbol{\eta}_{k+1} = (\mathbf{I} - \gamma \mathbf{H}) \boldsymbol{\eta}_k, \quad \begin{cases} a_{1,k+1} = (1 - \gamma)a_{1,k}, \\ a_{2,k+1} = (1 - 4\gamma)a_{2,k}. \end{cases}$$

DIMENSION / EDGE CHECK

Both modes shrink only for $0 < \gamma < 2/4 = 0.5$. In general the largest eigenvalue gives

$$0 < \gamma < 2/\lambda_{\max}.$$

STEP 2 · FINISH

Compare three rates:

γ	$1 - \gamma$	$1 - 4\gamma$	behavior
0.4	0.6	-0.6	both shrink
0.5	0.5	-1	boundary
0.6	0.4	-1.4	mode 2 grows

COMMON TRAP

A rate can be stable for the flat direction but unstable for the stiff direction. A negative factor means oscillation, not automatically divergence.

Eigenmode stability

BOARD DERIVATION

3 minutes · derive, check, explain

Question: For a concrete two-direction quadratic, which learning rates shrink both errors?

Given: $\mathbf{H} = \text{diag}(1, 4)$, $\mathbf{w}^* = \mathbf{0}$, and $\boldsymbol{\eta}_0 = (1, 1)^\top$. The second direction has four times the curvature.

STEP 1 · STRUCTURE

GD acts independently on the two coordinates:

$$\boldsymbol{\eta}_{k+1} = (\mathbf{I} - \gamma \mathbf{H}) \boldsymbol{\eta}_k, \quad \begin{cases} a_{1,k+1} = (1 - \gamma) a_{1,k}, \\ a_{2,k+1} = (1 - 4\gamma) a_{2,k}. \end{cases}$$

DIMENSION / EDGE CHECK

Both modes shrink only for $0 < \gamma < 2/4 = 0.5$. In general the largest eigenvalue gives

$$0 < \gamma < 2/\lambda_{\max}.$$

EXPLAIN IN ONE SENTENCE

With $\gamma = 0.4$, both errors shrink but the stiff direction flips sign; at 0.5 it never decays; above 0.5 it grows.

STEP 2 · FINISH

Compare three rates:

γ	$1 - \gamma$	$1 - 4\gamma$	behavior
0.4	0.6	-0.6	both shrink
0.5	0.5	-1	boundary
0.6	0.4	-1.4	mode 2 grows

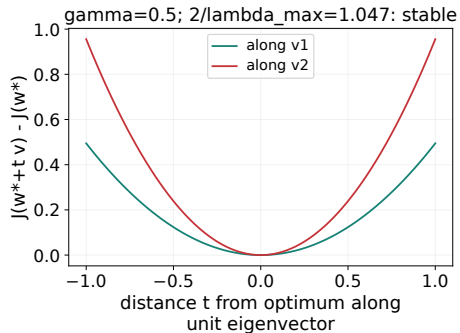
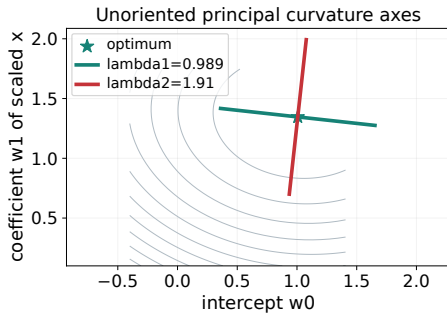
COMMON TRAP

A rate can be stable for the flat direction but unstable for the stiff direction. A negative factor means oscillation, not automatically divergence.

What the experiment shows

Curvature $\rightarrow$ Hessian eigen-directions

Which direction bends the loss most, and what happens exactly at the stability boundary?

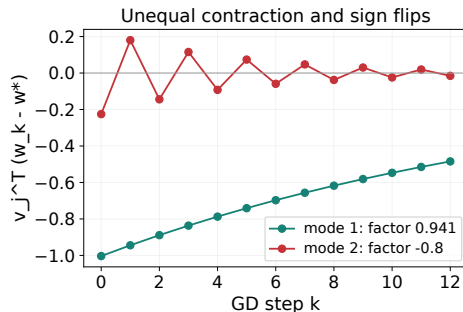
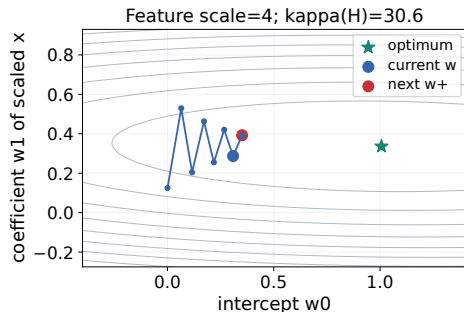


Conclusion: For $J = \|y - Xw\|^2 / (2N)$, $H = X^T X / N$. Its eigenvectors are unoriented curvature axes. Each mode multiplies by $1 - \gamma \lambda$. Stable: $0 < \gamma < 2/\lambda_{\max}$; equality flips the stiff mode without decay; above it a nonzero stiff mode grows.

What the experiment shows

Conditioning $\rightarrow$ unequal contraction and zig-zags

Why can stable full-gradient steps zig-zag while progress along the valley stays slow?



Conclusion: Unequal eigenvalues mean unequal contraction. Near the stability limit the stiff mode flips while the flat mode decays slowly, producing zig-zags. Increasing batch size cannot repair this full-gradient conditioning problem.

Why the mini-batch gradient is unbiased

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Show that uniform sampling without replacement preserves the full gradient in expectation.

Given: $\mathbf{g} = \frac{1}{N} \sum_{i=1}^N \mathbf{g}_i$, $\mathbf{g}_B = \frac{1}{B} \sum_{i \in B} \mathbf{g}_i$, and $|B| = B$.

Why the mini-batch gradient is unbiased

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Show that uniform sampling without replacement preserves the full gradient in expectation.

Given: $\mathbf{g} = \frac{1}{N} \sum_{i=1}^N \mathbf{g}_i$, $\mathbf{g}_{\mathcal{B}} = \frac{1}{B} \sum_{i \in \mathcal{B}} \mathbf{g}_i$, and $|\mathcal{B}| = B$.

STEP 1 · STRUCTURE

Introduce inclusion indicators $I_i = \mathbf{1}\{i \in \mathcal{B}\}$:

$$\mathbf{g}_{\mathcal{B}} = \frac{1}{B} \sum_{i=1}^N I_i \mathbf{g}_i, \quad \mathbb{E}[I_i] = \Pr(i \in \mathcal{B}) = \frac{B}{N}.$$

Why the mini-batch gradient is unbiased

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Show that uniform sampling without replacement preserves the full gradient in expectation.

Given: $\mathbf{g} = \frac{1}{N} \sum_{i=1}^N \mathbf{g}_i$, $\mathbf{g}_{\mathcal{B}} = \frac{1}{B} \sum_{i \in \mathcal{B}} \mathbf{g}_i$, and $|\mathcal{B}| = B$.

STEP 1 · STRUCTURE

Introduce inclusion indicators $I_i = \mathbf{1}\{i \in \mathcal{B}\}$:

$$\mathbf{g}_{\mathcal{B}} = \frac{1}{B} \sum_{i=1}^N I_i \mathbf{g}_i, \quad \mathbb{E}[I_i] = \Pr(i \in \mathcal{B}) = \frac{B}{N}.$$

STEP 2 · FINISH

Apply linearity of expectation:

$$\mathbb{E}[\mathbf{g}_{\mathcal{B}}] = \frac{1}{B} \sum_i \frac{B}{N} \mathbf{g}_i = \frac{1}{N} \sum_i \mathbf{g}_i = \mathbf{g}.$$

Why the mini-batch gradient is unbiased

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Show that uniform sampling without replacement preserves the full gradient in expectation.

Given: $\mathbf{g} = \frac{1}{N} \sum_{i=1}^N \mathbf{g}_i$, $\mathbf{g}_{\mathcal{B}} = \frac{1}{B} \sum_{i \in \mathcal{B}} \mathbf{g}_i$, and $|\mathcal{B}| = B$.

STEP 1 · STRUCTURE

Introduce inclusion indicators $I_i = \mathbf{1}\{i \in \mathcal{B}\}$:

$$\mathbf{g}_{\mathcal{B}} = \frac{1}{B} \sum_{i=1}^N I_i \mathbf{g}_i, \quad \mathbb{E}[I_i] = \Pr(i \in \mathcal{B}) = \frac{B}{N}.$$

DIMENSION / EDGE CHECK

At $B = N$, $\mathbf{g}_{\mathcal{B}} = \mathbf{g}$ exactly. For $B = 1$, the estimate is still unbiased but can have high variance.

STEP 2 · FINISH

Apply linearity of expectation:

$$\mathbb{E}[\mathbf{g}_{\mathcal{B}}] = \frac{1}{B} \sum_i \frac{B}{N} \mathbf{g}_i = \frac{1}{N} \sum_i \mathbf{g}_i = \mathbf{g}.$$

COMMON TRAP

Unbiased does not mean noiseless or faster. Without-replacement samples are dependent, so the variance is not obtained by pretending the I_i are independent.

Why the mini-batch gradient is unbiased

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Show that uniform sampling without replacement preserves the full gradient in expectation.

Given: $\mathbf{g} = \frac{1}{N} \sum_{i=1}^N \mathbf{g}_i$, $\mathbf{g}_{\mathcal{B}} = \frac{1}{B} \sum_{i \in \mathcal{B}} \mathbf{g}_i$, and $|\mathcal{B}| = B$.

STEP 1 · STRUCTURE

Introduce inclusion indicators $I_i = \mathbf{1}\{i \in \mathcal{B}\}$:

$$\mathbf{g}_{\mathcal{B}} = \frac{1}{B} \sum_{i=1}^N I_i \mathbf{g}_i, \quad \mathbb{E}[I_i] = \Pr(i \in \mathcal{B}) = \frac{B}{N}.$$

DIMENSION / EDGE CHECK

At $B = N$, $\mathbf{g}_{\mathcal{B}} = \mathbf{g}$ exactly. For $B = 1$, the estimate is still unbiased but can have high variance.

EXPLAIN IN ONE SENTENCE

The batch gradient is centered at the full gradient; batch size controls dispersion, not the center.

STEP 2 · FINISH

Apply linearity of expectation:

$$\mathbb{E}[\mathbf{g}_{\mathcal{B}}] = \frac{1}{B} \sum_i \frac{B}{N} \mathbf{g}_i = \frac{1}{N} \sum_i \mathbf{g}_i = \mathbf{g}.$$

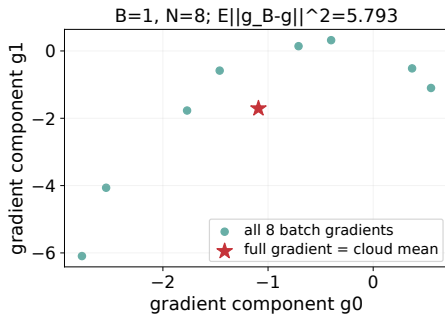
COMMON TRAP

Unbiased does not mean noiseless or faster. Without-replacement samples are dependent, so the variance is not obtained by pretending the I_i are independent.

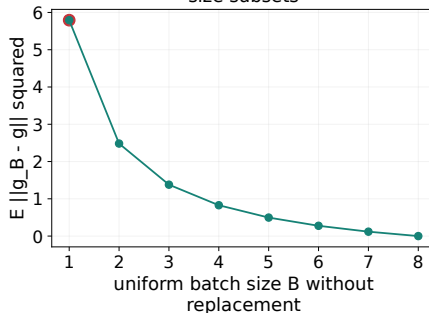
What the experiment shows

Sample gradients $\rightarrow$ an exact gradient cloud

Where should the cloud of all size-B batch gradients be centered, and what happens at $B=N$?



Exact dispersion across ALL fixed-size subsets



Conclusion: Every subset of size B is equally likely, sampled without replacement. Cloud mean = full gradient. $\text{Cov}(g_B) = (N-B) \sum_i (g_i - g)(g_i - g)^T / [BN(N-1)]$; $B=N$ has zero sampling variance. No universal exact $1/B$ law is claimed.

Stochastic Gradient Descent (SGD)

- **Sum Objectives:** In machine learning, most cost functions are formulated as a **sum** over the training examples, that is

$$J(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N J_n(\mathbf{w}) ,$$

Stochastic Gradient Descent (SGD)

- **Sum Objectives:** In machine learning, most cost functions are formulated as a **sum** over the training examples, that is

$$J(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N J_n(\mathbf{w}) ,$$

where J_n is the cost contributed by the n -th training example.

Stochastic Gradient Descent (SGD)

- **Sum Objectives:** In machine learning, most cost functions are formulated as a **sum** over the training examples, that is

$$J(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N J_n(\mathbf{w}) ,$$

where J_n is the cost contributed by the n -th training example.

- **The SGD Algorithm:** The **Stochastic Gradient Descent** (SGD) algorithm is given by the following update rule, at step t :

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \nabla J_n(\mathbf{w}^{(t)}) .$$

Stochastic Gradient Descent (SGD)

- **Sum Objectives:** In machine learning, most cost functions are formulated as a **sum** over the training examples, that is

$$J(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N J_n(\mathbf{w}) ,$$

where J_n is the cost contributed by the n -th training example.

- **The SGD Algorithm:** The **Stochastic Gradient Descent** (SGD) algorithm is given by the following update rule, at step t :

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \nabla J_n(\mathbf{w}^{(t)}) .$$

- **Theoretical Motivation:** If n is sampled uniformly from $\{1, \dots, N\}$, then

$$\mathbb{E}[\nabla J_n(\mathbf{w})] = \nabla J(\mathbf{w})$$

which is the true gradient direction.

- **Mini-batch SGD:** There is an intermediate version, using the update direction being

$$\mathbf{g} := \frac{1}{|\mathcal{B}|} \sum_{n \in \mathcal{B}} \nabla J_n(\mathbf{w}^{(t)})$$

- **Mini-batch SGD:** There is an intermediate version, using the update direction being

$$\mathbf{g} := \frac{1}{|\mathcal{B}|} \sum_{n \in \mathcal{B}} \nabla J_n(\mathbf{w}^{(t)})$$

again with

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \mathbf{g} .$$

- **Mini-batch SGD:** There is an intermediate version, using the update direction being

$$\mathbf{g} := \frac{1}{|\mathcal{B}|} \sum_{n \in \mathcal{B}} \nabla J_n(\mathbf{w}^{(t)})$$

again with

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \mathbf{g} .$$

- We randomly choose a subset $\mathcal{B} \subseteq [N]$ of the training examples.

- **Mini-batch SGD:** There is an intermediate version, using the update direction being

$$\mathbf{g} := \frac{1}{|\mathcal{B}|} \sum_{n \in \mathcal{B}} \nabla J_n(\mathbf{w}^{(t)})$$

again with

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \mathbf{g} .$$

- We randomly choose a subset $\mathcal{B} \subseteq [N]$ of the training examples. For each of these selected examples n , we compute the respective gradient ∇J_n , at the same current point $\mathbf{w}^{(t)}$.

- **Mini-batch SGD:** There is an intermediate version, using the update direction being

$$\mathbf{g} := \frac{1}{|\mathcal{B}|} \sum_{n \in \mathcal{B}} \nabla J_n(\mathbf{w}^{(t)})$$

again with

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \mathbf{g} .$$

- We randomly choose a subset $\mathcal{B} \subseteq [N]$ of the training examples. For each of these selected examples n , we compute the respective gradient ∇J_n , at the same current point $\mathbf{w}^{(t)}$.
- The batch computation can be expressed as vectorized matrix operations that GPUs execute efficiently.

- **Mini-batch SGD:** There is an intermediate version, using the update direction being

$$\mathbf{g} := \frac{1}{|\mathcal{B}|} \sum_{n \in \mathcal{B}} \nabla J_n(\mathbf{w}^{(t)})$$

again with

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \mathbf{g} .$$

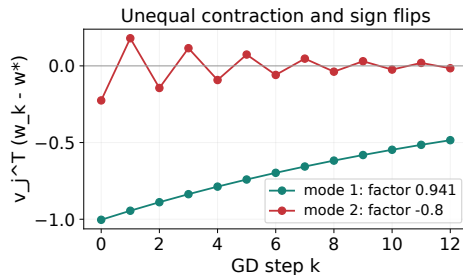
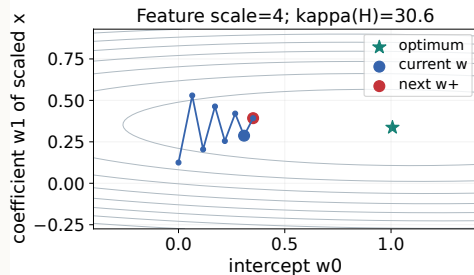
- We randomly choose a subset $\mathcal{B} \subseteq [N]$ of the training examples. For each of these selected examples n , we compute the respective gradient ∇J_n , at the same current point $\mathbf{w}^{(t)}$.
- The batch computation can be expressed as vectorized matrix operations that GPUs execute efficiently.
- In the extreme case $\mathcal{B} := [N]$, we obtain batch gradient descent, i.e. $\mathbf{g} = \nabla J$.

What the experiment shows

New optimization situation → method diagnosis

Which intervention addresses the cause: a smaller rate, feature normalization, or a larger batch?

New case: a feature is recorded in units four times larger; full-batch GD zig-zags. Would a bigger batch solve this?



Conclusion: No: it already uses the full batch. Inspect the Hessian spectrum, choose a stable rate for the new units, and normalize or precondition to reduce unequal contraction.

Table of Contents

1 Regression and Classification

- Regression
- Classification

2 Linear Regression

- Definition of Linear Regression
- Optimization Basics
- Gradient Descent (GD) variants for Linear Regression
- Normal Equations and Least Squares

Differential to normal equations

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Derive the normal equations and state what they mean geometrically.

Given: $J(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 / (2N)$ and $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w}$.

Differential to normal equations

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Derive the normal equations and state what they mean geometrically.

Given: $J(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 / (2N)$ and $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w}$.

STEP 1 · STRUCTURE

Use the differential already established:

$$dJ = \left(-\frac{1}{N} \mathbf{X}^\top \mathbf{r} \right)^\top d\mathbf{w}, \quad \nabla J(\mathbf{w}) = -\frac{1}{N} \mathbf{X}^\top \mathbf{r}.$$

Differential to normal equations

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Derive the normal equations and state what they mean geometrically.

Given: $J(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 / (2N)$ and $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w}$.

STEP 1 · STRUCTURE

Use the differential already established:

$$dJ = \left(-\frac{1}{N} \mathbf{X}^\top \mathbf{r} \right)^\top d\mathbf{w}, \quad \nabla J(\mathbf{w}) = -\frac{1}{N} \mathbf{X}^\top \mathbf{r}.$$

STEP 2 · FINISH

Set the gradient to zero and substitute $\mathbf{r}$:

$$\mathbf{X}^\top \mathbf{r} = \mathbf{0} \iff \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0} \iff \mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top \mathbf{y}.$$

Differential to normal equations

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Derive the normal equations and state what they mean geometrically.

Given: $J(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 / (2N)$ and $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w}$.

STEP 1 · STRUCTURE

Use the differential already established:

$$dJ = \left(-\frac{1}{N} \mathbf{X}^\top \mathbf{r} \right)^\top d\mathbf{w}, \quad \nabla J(\mathbf{w}) = -\frac{1}{N} \mathbf{X}^\top \mathbf{r}.$$

DIMENSION / EDGE CHECK

$\mathbf{X}^\top \mathbf{r} = \mathbf{0}$ contains D scalar orthogonality conditions. Because $\mathbf{H} = \mathbf{X}^\top \mathbf{X} / N \succeq 0$, stationarity is globally sufficient.

STEP 2 · FINISH

Set the gradient to zero and substitute $\mathbf{r}$:

$$\mathbf{X}^\top \mathbf{r} = \mathbf{0} \iff \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0} \iff \mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top \mathbf{y}.$$

COMMON TRAP

The normal equations need not identify a unique $\mathbf{w}$: rank deficiency makes $\mathbf{X}^\top \mathbf{X}$ singular while least-squares minimizers still exist.

Differential to normal equations

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Derive the normal equations and state what they mean geometrically.

Given: $J(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 / (2N)$ and $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w}$.

STEP 1 · STRUCTURE

Use the differential already established:

$$dJ = \left(-\frac{1}{N} \mathbf{X}^\top \mathbf{r} \right)^\top d\mathbf{w}, \quad \nabla J(\mathbf{w}) = -\frac{1}{N} \mathbf{X}^\top \mathbf{r}.$$

DIMENSION / EDGE CHECK

$\mathbf{X}^\top \mathbf{r} = \mathbf{0}$ contains D scalar orthogonality conditions. Because $\mathbf{H} = \mathbf{X}^\top \mathbf{X} / N \succeq 0$, stationarity is globally sufficient.

EXPLAIN IN ONE SENTENCE

At a least-squares solution, the residual is orthogonal to every design column; uniqueness of prediction does not imply uniqueness of parameters.

STEP 2 · FINISH

Set the gradient to zero and substitute $\mathbf{r}$:

$$\mathbf{X}^\top \mathbf{r} = \mathbf{0} \iff \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0} \iff \mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top \mathbf{y}.$$

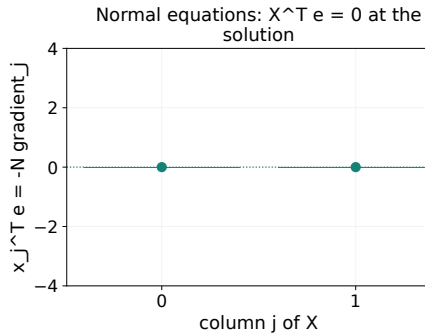
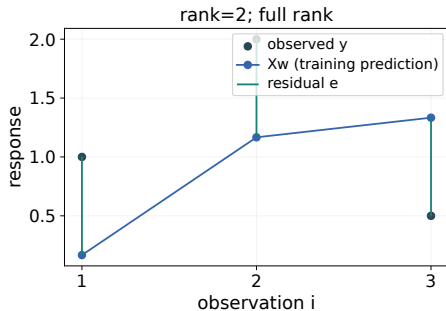
COMMON TRAP

The normal equations need not identify a unique $\mathbf{w}$: rank deficiency makes $\mathbf{X}^\top \mathbf{X}$ singular while least-squares minimizers still exist.

What the experiment shows

Zero gradient $\rightarrow$ normal equations

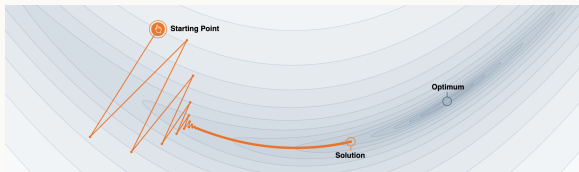
What equation must the residual satisfy when no parameter direction can lower half-MSE?



Conclusion: $dJ = -e^T X dw / N$, so gradient $J = -X^T e / N$. At a minimum $X^T e = 0$, equivalently $X^T X w = X^T y$. Positive semidefinite curvature makes stationarity sufficient, even if rank is deficient.

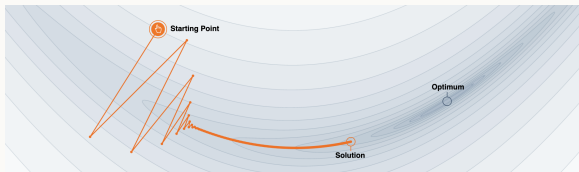
Motivation

Recall that GD for linear regression with MSE is an iterative method that is computationally flexible for large datasets.



Motivation

Recall that GD for linear regression with MSE is an iterative method that is computationally flexible for large datasets.



Can we compute the optimum of the cost function analytically?

- For linear regression with MSE, stationarity gives a linear system.

- For linear regression with MSE, stationarity gives a linear system.
- We can characterize an optimum **by solving this system**.

- For linear regression with MSE, stationarity gives a linear system.
- We can characterize an optimum **by solving this system**.
⇒ These equations are sometimes called the **normal equations**.

- For linear regression with MSE, stationarity gives a linear system.
- We can characterize an optimum **by solving this system**.
 - ⇒ These equations are sometimes called the **normal equations**.
 - ⇒ **Least squares** is the minimization problem; the normal equations characterize its solutions.

Recall that we use one-half of MSE (which has the same minimizers):

Recall that we use one-half of MSE (which has the same minimizers):

$$J(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 = \frac{1}{2N} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}), \quad (14)$$

where

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N, \quad \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1D} \\ x_{21} & x_{22} & \dots & x_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{ND} \end{bmatrix} \in \mathbb{R}^{N \times D}. \quad (15)$$

Recall that we use one-half of MSE (which has the same minimizers):

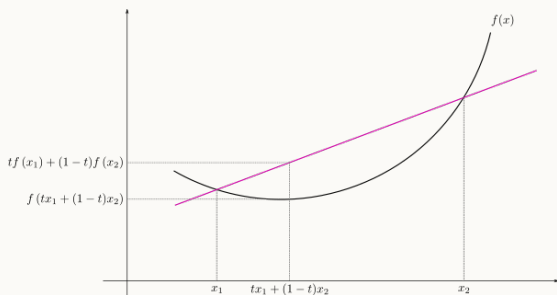
$$J(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 = \frac{1}{2N} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}), \quad (14)$$

where

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N, \quad \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1D} \\ x_{21} & x_{22} & \dots & x_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{ND} \end{bmatrix} \in \mathbb{R}^{N \times D}. \quad (15)$$

What is the meaning of finding the analytical solution?

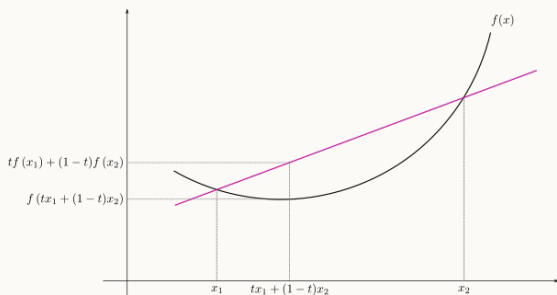
Two ingredients for the normal equations



- 1 Show that MSE is convex in $\mathbf{w}$.
- 2 Set the gradient to zero:

$$\nabla J(\mathbf{w}^*) = \mathbf{0}.$$

Two ingredients for the normal equations



- 1 Show that MSE is convex in $\mathbf{w}$.
- 2 Set the gradient to zero:

$$\nabla J(\mathbf{w}^*) = \mathbf{0}.$$

For a differentiable convex objective, this condition is sufficient for global optimality.

Derivation (step 1): MSE is convex in $\mathbf{w}$

$$J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2, \quad \nabla^2 J(\mathbf{w}) = \frac{1}{N} \mathbf{X}^\top \mathbf{X}. \quad (16)$$

Derivation (step 1): MSE is convex in $\mathbf{w}$

$$J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2, \quad \nabla^2 J(\mathbf{w}) = \frac{1}{N} \mathbf{X}^\top \mathbf{X}. \quad (16)$$

For every $\mathbf{v} \in \mathbb{R}^D$,

Derivation (step 1): MSE is convex in $\mathbf{w}$

$$J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2, \quad \nabla^2 J(\mathbf{w}) = \frac{1}{N} \mathbf{X}^\top \mathbf{X}. \quad (16)$$

For every $\mathbf{v} \in \mathbb{R}^D$,

$$\mathbf{v}^\top \nabla^2 J(\mathbf{w}) \mathbf{v} = \frac{1}{N} \mathbf{v}^\top \mathbf{X}^\top \mathbf{X} \mathbf{v} = \frac{1}{N} \|\mathbf{X}\mathbf{v}\|_2^2 \geq 0. \quad (17)$$

Therefore the Hessian is positive semidefinite and J is convex in $\mathbf{w}$.

Derivation (step 2): finding the minimum of a convex function

By taking the gradient of $J(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 = \frac{1}{2N} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w})$, we have

$$\nabla J(\mathbf{w}) = -\frac{1}{N} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}). \quad (18)$$

Derivation (step 2): finding the minimum of a convex function

By taking the gradient of $J(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 = \frac{1}{2N} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w})$, we have

$$\nabla J(\mathbf{w}) = -\frac{1}{N} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}). \quad (18)$$

At an unconstrained minimum, differentiability gives $\nabla J(\mathbf{w}^*) = \mathbf{0}$; convexity makes this condition sufficient.

Derivation (step 2): finding the minimum of a convex function

By taking the gradient of $J(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 = \frac{1}{2N} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w})$, we have

$$\nabla J(\mathbf{w}) = -\frac{1}{N} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}). \quad (18)$$

At an unconstrained minimum, differentiability gives $\nabla J(\mathbf{w}^*) = \mathbf{0}$; convexity makes this condition sufficient. we can get the [normal equations for linear regression](#):

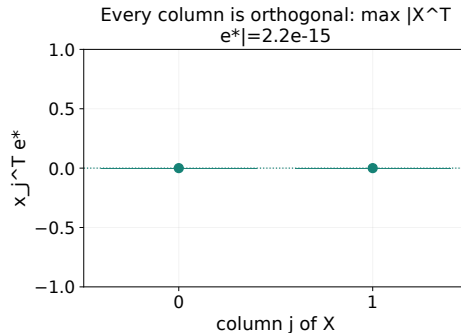
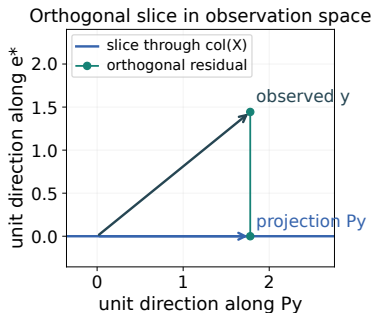
$$\mathbf{X}^\top \underbrace{(\mathbf{y} - \mathbf{X}\mathbf{w}^*)}_{\text{residual}} = \mathbf{0}, \quad (19)$$

where the residual $\mathbf{r}^* := \mathbf{y} - \mathbf{X}\mathbf{w}^*$ is orthogonal to every column of $\mathbf{X}$.

What the experiment shows

Normal equations $\rightarrow$ orthogonal projection

Why must the fitted residual be perpendicular to every column of the design?



Conclusion: The residual is orthogonal to EVERY column: $X^T e^* = 0$. The unique training prediction is the orthogonal projection P_y , even if weights are nonunique. The plot is an orthonormal slice through P_y and e^* .

Geometric interpretation of Normal Equations

Definition 5 (Span of a set of vectors)

The **span** of a set of vectors, $\{\mathbf{x}_1, \dots, \mathbf{x}_k\}$, is the set of all possible **linear combinations** of these vectors; i.e.

$$\text{span}\{\mathbf{x}_1, \dots, \mathbf{x}_k\} = \{\alpha_1 \mathbf{x}_1 + \dots + \alpha_k \mathbf{x}_k \mid \alpha_1, \dots, \alpha_k \in \mathbb{R}\}.$$

Geometric interpretation of Normal Equations

Definition 5 (Span of a set of vectors)

The **span** of a set of vectors, $\{\mathbf{x}_1, \dots, \mathbf{x}_k\}$, is the set of all possible **linear combinations** of these vectors; i.e. $\text{span}\{\mathbf{x}_1, \dots, \mathbf{x}_k\} = \{\alpha_1 \mathbf{x}_1 + \dots + \alpha_k \mathbf{x}_k \mid \alpha_1, \dots, \alpha_k \in \mathbb{R}\}$.

- The **span** of $\mathbf{X} \in \mathbb{R}^{N \times D}$ is the space spanned by the columns of $\mathbf{X}$

$$\mathcal{S} := \text{span}(\mathbf{X}) = \{\mathbf{u} := \mathbf{X}\mathbf{w} \mid \mathbf{w} \in \mathbb{R}^D\}$$

Geometric interpretation of Normal Equations

Definition 5 (Span of a set of vectors)

The **span** of a set of vectors, $\{\mathbf{x}_1, \dots, \mathbf{x}_k\}$, is the set of all possible **linear combinations** of these vectors; i.e. $\text{span}\{\mathbf{x}_1, \dots, \mathbf{x}_k\} = \{\alpha_1 \mathbf{x}_1 + \dots + \alpha_k \mathbf{x}_k \mid \alpha_1, \dots, \alpha_k \in \mathbb{R}\}$.

- The **span** of $\mathbf{X} \in \mathbb{R}^{N \times D}$ is the space spanned by the columns of $\mathbf{X}$

$$\mathcal{S} := \text{span}(\mathbf{X}) = \{\mathbf{u} := \mathbf{X}\mathbf{w} \mid \mathbf{w} \in \mathbb{R}^D\}$$

Which element $\mathbf{u}$ of $\text{span}(\mathbf{X})$ shall we take such that the normal equation $\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) = 0$?

Geometric interpretation of Normal Equations

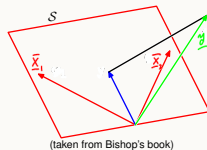
Definition 5 (Span of a set of vectors)

The **span** of a set of vectors, $\{\mathbf{x}_1, \dots, \mathbf{x}_k\}$, is the set of all possible **linear combinations** of these vectors; i.e. $\text{span}\{\mathbf{x}_1, \dots, \mathbf{x}_k\} = \{\alpha_1 \mathbf{x}_1 + \dots + \alpha_k \mathbf{x}_k \mid \alpha_1, \dots, \alpha_k \in \mathbb{R}\}$.

- The **span** of $\mathbf{X} \in \mathbb{R}^{N \times D}$ is the space spanned by the columns of $\mathbf{X}$

$$\mathcal{S} := \text{span}(\mathbf{X}) = \{\mathbf{u} := \mathbf{X}\mathbf{w} \mid \mathbf{w} \in \mathbb{R}^D\}$$

Which element $\mathbf{u}$ of $\text{span}(\mathbf{X})$ shall we take such that the normal equation $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$?



Geometric interpretation of Normal Equations

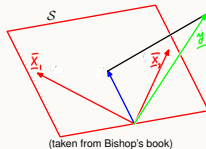
Definition 5 (Span of a set of vectors)

The **span** of a set of vectors, $\{\mathbf{x}_1, \dots, \mathbf{x}_k\}$, is the set of all possible **linear combinations** of these vectors; i.e. $\text{span}\{\mathbf{x}_1, \dots, \mathbf{x}_k\} = \{\alpha_1 \mathbf{x}_1 + \dots + \alpha_k \mathbf{x}_k \mid \alpha_1, \dots, \alpha_k \in \mathbb{R}\}$.

- The **span** of $\mathbf{X} \in \mathbb{R}^{N \times D}$ is the space spanned by the columns of $\mathbf{X}$

$$\mathcal{S} := \text{span}(\mathbf{X}) = \{\mathbf{u} := \mathbf{X}\mathbf{w} \mid \mathbf{w} \in \mathbb{R}^D\}$$

Which element $\mathbf{u}$ of $\text{span}(\mathbf{X})$ shall we take such that the normal equation $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$?



From $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$, we have:

Geometric interpretation of Normal Equations

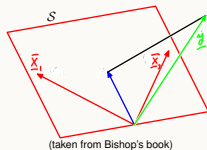
Definition 5 (Span of a set of vectors)

The **span** of a set of vectors, $\{\mathbf{x}_1, \dots, \mathbf{x}_k\}$, is the set of all possible **linear combinations** of these vectors; i.e. $\text{span}\{\mathbf{x}_1, \dots, \mathbf{x}_k\} = \{\alpha_1 \mathbf{x}_1 + \dots + \alpha_k \mathbf{x}_k \mid \alpha_1, \dots, \alpha_k \in \mathbb{R}\}$.

- The **span** of $\mathbf{X} \in \mathbb{R}^{N \times D}$ is the space spanned by the columns of $\mathbf{X}$

$$\mathcal{S} := \text{span}(\mathbf{X}) = \{\mathbf{u} := \mathbf{X}\mathbf{w} \mid \mathbf{w} \in \mathbb{R}^D\}$$

Which element $\mathbf{u}$ of $\text{span}(\mathbf{X})$ shall we take such that the normal equation $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$?



(taken from Bishop's book)

From $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$, we have:

- the optimum choice for $\mathbf{u}$, i.e. $\mathbf{u}^*$, requires $\mathbf{y} - \mathbf{u}^*$ to be orthogonal to $\text{span}(\mathbf{X})$.

Geometric interpretation of Normal Equations

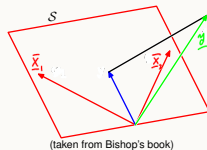
Definition 5 (Span of a set of vectors)

The **span** of a set of vectors, $\{\mathbf{x}_1, \dots, \mathbf{x}_k\}$, is the set of all possible **linear combinations** of these vectors; i.e. $\text{span}\{\mathbf{x}_1, \dots, \mathbf{x}_k\} = \{\alpha_1 \mathbf{x}_1 + \dots + \alpha_k \mathbf{x}_k \mid \alpha_1, \dots, \alpha_k \in \mathbb{R}\}$.

- The **span** of $\mathbf{X} \in \mathbb{R}^{N \times D}$ is the space spanned by the columns of $\mathbf{X}$

$$\mathcal{S} := \text{span}(\mathbf{X}) = \{\mathbf{u} := \mathbf{X}\mathbf{w} \mid \mathbf{w} \in \mathbb{R}^D\}$$

Which element $\mathbf{u}$ of $\text{span}(\mathbf{X})$ shall we take such that the normal equation $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$?



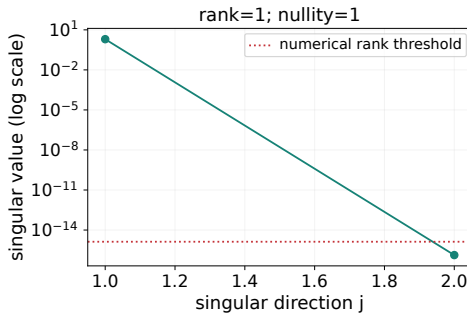
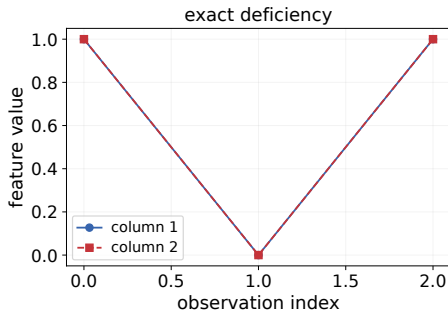
From $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$, we have:

- the optimum choice for $\mathbf{u}$, i.e. $\mathbf{u}^*$, requires $\mathbf{y} - \mathbf{u}^*$ to be orthogonal to $\text{span}(\mathbf{X})$.
- $\mathbf{u}^*$ should be equal to *the projection of $\mathbf{y}$ onto $\text{span}(\mathbf{X})$* .

What the experiment shows

Feature dependence $\rightarrow$ rank deficiency

Do nearly identical features and exactly identical features have the same mathematical consequences?



Conclusion: At $\epsilon=0$ the columns coincide: exact deficiency, rank 1, nullity 1. At $\epsilon=1e-5$ the columns remain independent: rank 2, nullity 0. Near dependence means sensitivity, not exact nonuniqueness.

Null shift, same prediction

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Track predictions, residuals, and loss after moving in a null-space direction.

Given: $X\mathbf{v} = \mathbf{0}$, $t \in \mathbb{R}$, and $\tilde{\mathbf{w}} = \mathbf{w} + t\mathbf{v}$.

Null shift, same prediction

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Track predictions, residuals, and loss after moving in a null-space direction.

Given: $X\mathbf{v} = \mathbf{0}$, $t \in \mathbb{R}$, and $\tilde{\mathbf{w}} = \mathbf{w} + t\mathbf{v}$.

STEP 1 · STRUCTURE

First compare training predictions:

$$X\tilde{\mathbf{w}} = X(\mathbf{w} + t\mathbf{v}) = X\mathbf{w} + tX\mathbf{v} = X\mathbf{w}.$$

Therefore $\tilde{\mathbf{r}} = \mathbf{y} - X\tilde{\mathbf{w}} = \mathbf{r}$.

Null shift, same prediction

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Track predictions, residuals, and loss after moving in a null-space direction.

Given: $X\mathbf{v} = \mathbf{0}$, $t \in \mathbb{R}$, and $\tilde{\mathbf{w}} = \mathbf{w} + t\mathbf{v}$.

STEP 1 · STRUCTURE

First compare training predictions:

$$X\tilde{\mathbf{w}} = X(\mathbf{w} + t\mathbf{v}) = X\mathbf{w} + tX\mathbf{v} = X\mathbf{w}.$$

Therefore $\tilde{\mathbf{r}} = \mathbf{y} - X\tilde{\mathbf{w}} = \mathbf{r}$.

STEP 2 · FINISH

The objective is unchanged:

$$J(\tilde{\mathbf{w}}) = \frac{1}{2N} \|\tilde{\mathbf{r}}\|_2^2 = \frac{1}{2N} \|\mathbf{r}\|_2^2 = J(\mathbf{w}).$$

Thus one minimizer generates an affine family.

Null shift, same prediction

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Track predictions, residuals, and loss after moving in a null-space direction.

Given: $X\mathbf{v} = \mathbf{0}$, $t \in \mathbb{R}$, and $\tilde{\mathbf{w}} = \mathbf{w} + t\mathbf{v}$.

STEP 1 · STRUCTURE

First compare training predictions:

$$X\tilde{\mathbf{w}} = X(\mathbf{w} + t\mathbf{v}) = X\mathbf{w} + tX\mathbf{v} = X\mathbf{w}.$$

Therefore $\tilde{\mathbf{r}} = \mathbf{y} - X\tilde{\mathbf{w}} = \mathbf{r}$.

DIMENSION / EDGE CHECK

All minimizers form $\mathbf{w}^\dagger + \text{null}(X)$. If $\text{null}(X) = \{\mathbf{0}\}$, the least-squares parameter is unique.

STEP 2 · FINISH

The objective is unchanged:

$$J(\tilde{\mathbf{w}}) = \frac{1}{2N} \|\tilde{\mathbf{r}}\|_2^2 = \frac{1}{2N} \|\mathbf{r}\|_2^2 = J(\mathbf{w}).$$

Thus one minimizer generates an affine family.
COMMON TRAP

Equal training predictions do not force equal predictions at a new row $\mathbf{x}_{\text{new}}^\top$ unless $\mathbf{x}_{\text{new}}^\top \mathbf{v} = 0$.

Null shift, same prediction

BOARD DERIVATION

3 minutes · derive, check, explain

Question: Track predictions, residuals, and loss after moving in a null-space direction.

Given: $X\mathbf{v} = \mathbf{0}$, $t \in \mathbb{R}$, and $\tilde{\mathbf{w}} = \mathbf{w} + t\mathbf{v}$.

STEP 1 · STRUCTURE

First compare training predictions:

$$X\tilde{\mathbf{w}} = X(\mathbf{w} + t\mathbf{v}) = X\mathbf{w} + tX\mathbf{v} = X\mathbf{w}.$$

Therefore $\tilde{\mathbf{r}} = \mathbf{y} - X\tilde{\mathbf{w}} = \mathbf{r}$.

DIMENSION / EDGE CHECK

All minimizers form $\mathbf{w}^\dagger + \text{null}(X)$. If $\text{null}(X) = \{\mathbf{0}\}$, the least-squares parameter is unique.

EXPLAIN IN ONE SENTENCE

A null-space shift changes coefficients but not training predictions, residuals, or loss; identifiability is a separate question.

STEP 2 · FINISH

The objective is unchanged:

$$J(\tilde{\mathbf{w}}) = \frac{1}{2N} \|\tilde{\mathbf{r}}\|_2^2 = \frac{1}{2N} \|\mathbf{r}\|_2^2 = J(\mathbf{w}).$$

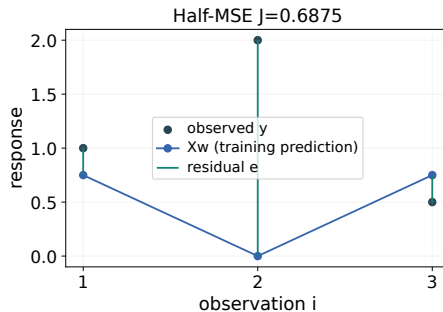
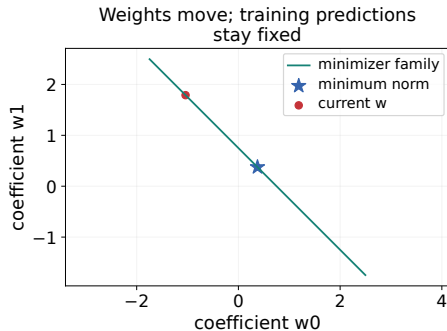
Thus one minimizer generates an affine family.
COMMON TRAP

Equal training predictions do not force equal predictions at a new row $\mathbf{x}_{\text{new}}^\top$ unless $\mathbf{x}_{\text{new}}^\top \mathbf{v} = 0$.

What the experiment shows

Null-space family $\rightarrow$ identifiable predictions

Can the coefficients move while every training prediction and the loss stay fixed?

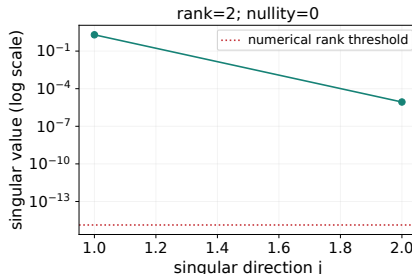
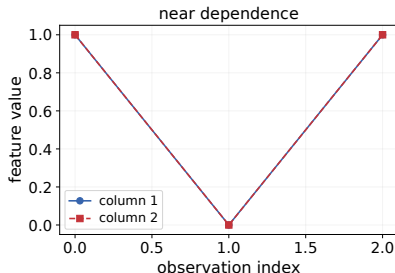


Conclusion: Moving w_{dagger} by $t v$ with $Xv=0$ changes the weights but leaves Xw and half-MSE fixed. Duplicate features identify only the coefficient sum. Full column rank has nullity 0, so no nonzero null direction exists.

What the experiment shows

Inverse formula $\rightarrow$ numerical solver choice

Why might a valid inverse formula still be a poor way to compute least squares?



Conclusion: The inverse formula requires full column rank; minimizers still exist at deficient rank. Favor a least-squares routine based on QR or SVD. Mathematical uniqueness and numerical stability are different questions.

Continue to synthesis Compare inverse, QR, and SVD

Least Squares

We need to solve the linear system of the normal equation $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$, where

$$\mathbf{X}^\top \mathbf{y} = \underbrace{\mathbf{X}^\top \mathbf{X}}_{\text{Gram matrix}} \mathbf{w}^* \quad (20)$$

Least Squares

We need to solve the linear system of the normal equation $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$, where

$$\mathbf{X}^\top \mathbf{y} = \underbrace{\mathbf{X}^\top \mathbf{X}}_{\text{Gram matrix}} \mathbf{w}^* \quad (20)$$

If the Gram matrix is invertible,

Least Squares

We need to solve the linear system of the normal equation $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$, where

$$\mathbf{X}^\top \mathbf{y} = \underbrace{\mathbf{X}^\top \mathbf{X}}_{\text{Gram matrix}} \mathbf{w}^* \quad (20)$$

If the Gram matrix is invertible, we can multiply the normal equation by the inverse of the Gram matrix from the left:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}, \quad (21)$$

Least Squares

We need to solve the linear system of the normal equation $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$, where

$$\mathbf{X}^\top \mathbf{y} = \underbrace{\mathbf{X}^\top \mathbf{X}}_{\text{Gram matrix}} \mathbf{w}^* \quad (20)$$

If the Gram matrix is invertible, we can multiply the normal equation by the inverse of the Gram matrix from the left:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}, \quad (21)$$

which is the unique minimizer when $\mathbf{X}$ has full column rank.

Least Squares

We need to solve the linear system of the normal equation $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$, where

$$\mathbf{X}^\top \mathbf{y} = \underbrace{\mathbf{X}^\top \mathbf{X}}_{\text{Gram matrix}} \mathbf{w}^* \quad (20)$$

If the Gram matrix is invertible, we can multiply the normal equation by the inverse of the Gram matrix from the left:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}, \quad (21)$$

which is the unique minimizer when $\mathbf{X}$ has full column rank.

We can use this model to predict a new value for an unseen datapoint (test point) $\mathbf{x}_m$:

$$\hat{y}_m := \mathbf{x}_m^\top \mathbf{w}^* = \mathbf{x}_m^\top (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (22)$$

Least Squares

We need to solve the linear system of the normal equation $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$, where

$$\mathbf{X}^\top \mathbf{y} = \underbrace{\mathbf{X}^\top \mathbf{X}}_{\text{Gram matrix}} \mathbf{w}^* \quad (20)$$

If the Gram matrix is invertible, we can multiply the normal equation by the inverse of the Gram matrix from the left:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}, \quad (21)$$

which is the unique minimizer when $\mathbf{X}$ has full column rank.

We can use this model to predict a new value for an unseen datapoint (test point) $\mathbf{x}_m$:

$$\hat{y}_m := \mathbf{x}_m^\top \mathbf{w}^* = \mathbf{x}_m^\top (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (22)$$

Remark 6

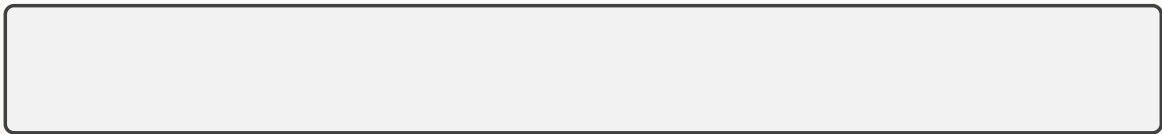
The inverse formula requires full column rank. Numerically, prefer a least-squares routine based on QR or SVD instead of explicitly forming $(\mathbf{X}^\top \mathbf{X})^{-1}$.

Rank deficiency and ill-conditioning

Unfortunately, in practice, $\mathbf{X} \in \mathbb{R}^{N \times D}$ is often **rank deficient**.

Rank deficiency and ill-conditioning

Unfortunately, in practice, $\mathbf{X} \in \mathbb{R}^{N \times D}$ is often **rank deficient**.
For example,



Rank deficiency and ill-conditioning

Unfortunately, in practice, $\mathbf{X} \in \mathbb{R}^{N \times D}$ is often **rank deficient**.

For example,

- If $D > N$ (namely over-parameterized), we always have $\text{rank}(\mathbf{X}) < D$

Rank deficiency and ill-conditioning

Unfortunately, in practice, $\mathbf{X} \in \mathbb{R}^{N \times D}$ is often **rank deficient**.

For example,

- If $D > N$ (namely over-parameterized), we always have $\text{rank}(\mathbf{X}) < D$ (since row rank = col. rank)

Rank deficiency and ill-conditioning

Unfortunately, in practice, $\mathbf{X} \in \mathbb{R}^{N \times D}$ is often **rank deficient**.

For example,

- If $D > N$ (namely over-parameterized), we always have $\text{rank}(\mathbf{X}) < D$ (since row rank = col. rank)
- If $D \leq N$, but some of the columns $\mathbf{x}_{:d}$ are (nearly) collinear

Rank deficiency and ill-conditioning

Unfortunately, in practice, $\mathbf{X} \in \mathbb{R}^{N \times D}$ is often **rank deficient**.

For example,

- If $D > N$ (namely over-parameterized), we always have $\text{rank}(\mathbf{X}) < D$ (since row rank = col. rank)
- If $D \leq N$, but some of the columns $\mathbf{x}_{:d}$ are (nearly) collinear
 $\Rightarrow \mathbf{X}$ is ill-conditioned, leading to numerical issues when solving the linear system.

Rank deficiency and ill-conditioning

Unfortunately, in practice, $\mathbf{X} \in \mathbb{R}^{N \times D}$ is often **rank deficient**.

For example,

- If $D > N$ (namely over-parameterized), we always have $\text{rank}(\mathbf{X}) < D$ (since row rank = col. rank)
- If $D \leq N$, but some of the columns $\mathbf{x}_{:d}$ are (nearly) collinear
 $\Rightarrow \mathbf{X}$ is ill-conditioned, leading to numerical issues when solving the linear system.

Can we solve least squares if $\mathbf{X}$ is **rank deficient**?

Rank deficiency and ill-conditioning

Unfortunately, in practice, $\mathbf{X} \in \mathbb{R}^{N \times D}$ is often **rank deficient**.

For example,

- If $D > N$ (namely over-parameterized), we always have $\text{rank}(\mathbf{X}) < D$ (since row rank = col. rank)
- If $D \leq N$, but some of the columns $\mathbf{x}_{:d}$ are (nearly) collinear
 $\Rightarrow \mathbf{X}$ is ill-conditioned, leading to numerical issues when solving the linear system.

Can we solve least squares if $\mathbf{X}$ is **rank deficient**?

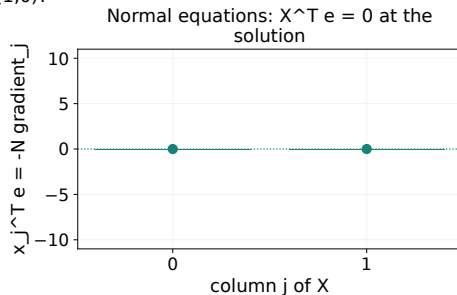
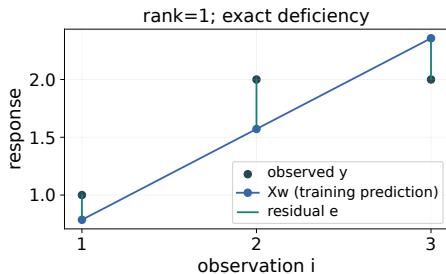
Yes. A least-squares solver can return a minimizer, e.g., `np.linalg.lstsq(X, y)`; with rank deficiency, the minimizer need not be unique.

What the experiment shows

New design matrix $\rightarrow$ synthesis and diagnosis

For an unfamiliar design, which quantities are unique and what computation would you trust?

New design: two sensors report the same feature, $X = \begin{bmatrix} 1 & 1 \\ 2 & 2 \\ 3 & 3 \end{bmatrix}$. Can training data identify their separate effects or predict at the new row $(1,0)$?



Conclusion: Rank is 1, nullity is 1. Training predictions are unique but only the sum of the two coefficients is identifiable. The new row $(1,0)$ distinguishes null-shifted solutions, so its prediction needs an additional assumption or measurement.

One model, four connected views:

- **Data:** predictions create residuals
- **Objective:** a loss says which residuals matter most
- **Optimization:** GD and SGD move parameters using gradients
- **Geometry:** least squares projects $\mathbf{y}$ onto $\text{col}(\mathbf{X})$

Next lecture:

- The probabilistic interpretation of Linear Regression
- Maximum Likelihood Estimation (MLE)
- Over-fitting and Under-fitting
- Polynomial Regression, Ridge Regression, and Lasso